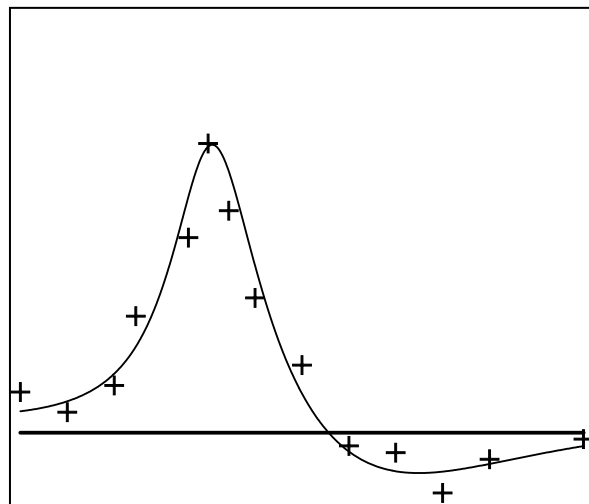


The HP Suite

**PEST_HP Compatible Programs for
Highly Parallelized
Computing Environments**

Manual for Version 18



Watermark Numerical Computing

February, 2025

Table of Contents

1. Introduction	1
1.1 General.....	1
1.2 Terminology	1
1.3 Using a HP Suite Program	2
1.4 Variables that Control Run Management	2
1.5 Run Management Output Files	2
1.6 Secondary Parameters	3
1.7 Observation Penalties	3
2. JACTEST_HP.....	4
3. CMAES_HP	5
3.1 General.....	5
3.2 Running CMAES_HP	5
3.3 Some CMAES_HP Details	6
3.3.1 Starting and Stopping.....	6
3.3.2 Extra Output Files.....	6
3.3.3 Forgiving Run Failure.....	7
3.4 File Parameters	7
3.4.1 General.....	7
3.4.2 Defining File Parameters.....	8
3.4.3 Supplying Values for File Parameters	10
3.4.4 Calculating the Objective Function	11
3.4.5 CMAES_HP Output.....	11
3.4.6 The Optional System Command	13
3.4.7 Some Other Implementation Details	13
3.5 Secondary Parameters	13
4. RSI_HP	15
4.1. Introduction	15
4.2 Running RSI_HP.....	15
4.3 Theory	16
4.3.1 Background	16
4.3.2 Basic Equations	16
4.4 Some Algorithmic Details.....	18
4.4.1 Parameter Transformations.....	18

4.4.2 The Marquardt Lambda	19
4.4.3 Supplying Realizations to RSI_HP.....	19
4.4.4 Using RSI_HP to Generate Observation Realisations.....	20
4.4.5 Supplying Observations Externally.....	20
4.4.6 Prior Information	21
4.4.7 Covariance Matrices.....	21
4.4.8 Adjustment Constraints	21
4.4.9 Parameter Bounds	22
4.4.10 Objective Functions	23
4.4.11 Upgrade Rejection	23
4.4.12 Realization Abandonment	24
4.4.13 Diversity Regularization – a Simple Option.....	24
4.4.14 Secondary Parameter Realizations	25
4.4.15 Supplementary Parameter Realizations	26
4.4.16 Multiple Data Assimilation.....	28
4.4.17 Termination Criteria.....	28
4.5 Regularisation	29
4.5.1 Regularisation for Ensemble Uniformity.....	29
4.5.2 Regularisation for Solution Divergence	31
4.6 RSI_HP Outputs.....	32
4.6.1 Screen.....	32
4.6.2 Run Record File	32
4.6.3 Ensemble Output Files	32
4.6.4 Phi Output Files.....	32
4.6.5 Relating File and Screen Output	33
4.7 RSI Control Variables.....	34
4.7.1 PEST Control File	34
4.7.2 Traditional PEST Control Variables	34
4.7.3 RSI Section.....	35
4.8 Supporting Utilities	41
5. Ensemble Space Inversion.....	42
5.1 Introduction	42
5.2 Concepts.....	42
5.2.1 Ensemble Smoothers	42
5.2.2 ENSI	43

5.2.3 Regularised Inversion	45
5.2.4 Two References	46
5.3 Practicalities	46
5.3.1 General	46
5.3.2 PEST Control File	47
5.3.3 Parameter Uncertainty File	47
5.3.4 ENSI Specification File	47
5.4 ENSIPREP	49
5.4.1 General	49
5.4.2 Running ENSIPREP	50
5.5 ENSIMOD	52
5.5.1 General	52
5.5.2 Running ENSIMOD	52
5.5.3 Parameter Value File	52
5.6 Some Important Workflow Elements	53
5.6.1 Posing the Inverse Problem	53
5.6.2 Number of Realisations	54
5.6.3 Running PEST	54
5.6.4 After Running PEST	55
5.7 Posterior Linear Uncertainty Analysis	56
5.7.1 General Linear Analysis	56
5.7.2 Posterior Parameter Uncertainties: A Simple Workflow	57
5.7.3 Running POSTENSIUNC	58
5.8 Summary of Workflow	59
5.8.1 General	59
5.8.2 Final Words	60
5.9 PCA Inversion	61
5.9.1 General	61
5.9.2 Theory	62
5.9.3 Setup	62
5.9.4 The Workflow	64
5.9.5 Which to Use?	64
5.9.6 Efficiency vs Bias	64
6. References	68

1. Introduction

1.1 General

“HP” stands for “highly parallelized” or “high performance”. The HP-suite is comprised of a growing number of programs, some of which are counterparts to programs of the same name in the PEST suite, and some of which do not have PEST suite counterparts. All of them read a PEST control file. All of them require that a model be run many times. All of them provide a non-intrusive interface with a model using template and instruction files.

This manual describes JACTEST_HP, CMAES_HP and RSI_HP. PEST_HP, as well as two programs that were written specifically to support the use of PEST_HP (namely PWHISP_HP and PCOST_HP), are described in a separate manual.

All members of the HP suite parallelize model runs using a similar protocol to that employed by BEOPEST. These runs can be carried out on one computer, or over many computers. While no member of the HP suite has been programmed to undertake serial model runs, there is no reason why it cannot supervise a single stream of model runs on a single machine. The numerical algorithms implemented by members of the HP suite readily adapt to whatever parallelization environment is available to them.

1.2 Terminology

The “master/slave” terminology employed by BEOPEST has been replaced by a new nomenclature. The “master” is now the “manager”. A “slave” is now an “agent”. All members of the HP suite employ the same agent; this is named AGENT_HP.

A manager has two roles. Firstly, it undertakes the set of numerical tasks, and implements the set of numerical algorithms, which define that particular member of the HP suite. Secondly, it commissions model runs as they are required by that algorithm. Each of these model runs is supervised by a local agent that runs on the same computer as that on which the model is run. Many agents can exist on one or many machines. Whenever a model run is required, the manager assigns that run to the fastest available agent, and transmits to that agent the parameters that the model must use on that run. The local agent then writes model input files which feature these parameters using local copies of template files of these model input files. Then it runs the model. When the model has finished running, the local agent reads pertinent numbers from model output files using local copies of instruction files. It then transmits these numbers back to the manager. Meanwhile, the manager accommodates late and overdue model runs, as well as agents that go missing.

The software that actually manages model runs is not a discrete executable program. Rather it is a set of functions and subroutines that are called by all HP suite programs. Hence control variables featured in a PEST control file which govern run management are the same for all of these programs.

Each agent must operate in a separate folder. If desired, one of these folders can be that used by the HP suite manager. This folder must contain a PEST control file, all template and instruction files that are cited in the PEST control file, and all files that are required to run the model. See documentation of BEOPEST for details.

The agent must have the same version number as the manager. If this is not the case, the manager will detect this incompatibility and cease execution with an appropriate error message.

1.3 Using a HP Suite Program

Use of a member of the HP suite is very similar to that of BEOPEST. To run the manager based on the contents of a PEST control file named *case.pst*, start that program using a command that bears the name of the manager. For example:

```
jactest_hp case /h :nnnn
```

```
cmaes_hp case /h :nnnn
```

```
rsi_hp case /h :nnnn
```

In each of these examples, *nnnn* is a port number (for example 4004). Note the space character between the “/h” string and the following colon.

Unlike BEOPEST, a member of the HP suite cannot operate as both a manager and an agent. Instead, members of the HP suite employ a common agent. This common agent is AGENT_HP. To run AGENT_HP, use the command

```
agent_hp case /h hostname:nnnn
```

where *hostname* is the name of the machine on which the manager is running, and *nnnn* is the port number employed by the manager (as provided on the manager’s command line). Alternatively use the IP address of the machine on which the manager is running instead of *hostname*. (Only TCP/IP version 4 is supported at the time of writing.)

Agents can be initiated before or after initiation of the manager.

1.4 Variables that Control Run Management

A number of variables that guide parallel run management can optionally be provided in the “control data” section of a PEST control file used by members of the HP suite. These are:

- HARDSTOPHOURS,
- SOFTSTOPHOURS,
- RUN_SLOW_FAC,
- RUN_ABANDON_FAC, and
- WIN_MRUN_HOURS.

As stated above, all members of the HP suite use the same run manager, and all members of the HP suite read a PEST control file. Hence all of the above variables can be supplied in a PEST control file that is used by any member of the HP suite. The manner in which these variables control parallel run management is described in the accompanying manual for PEST_HP.

1.5 Run Management Output Files

Suppose that a member of the HP suite reads a PEST control file named *case.pst*. Most programs of the HP suite record a run record file named *case.rec*, together with other files which are specific to the task that each of them undertakes. Most of them also write the following files; these contain information that is gathered by their common run manager.

- *case.rmr*, which records the communications history between the run manger and all of its run agents;
- *case.per*, which lists parameter values associated with failed model runs; and
- *case.rme*, which records run agent efficiencies.

See documentation of PEST_HP for details of the contents of these files.

Files *pest###.dap* and *pest###.dao* which are used for PEST_HP parallel run management are also used for management of parallel model runs by the members of the HP suite.

1.6 Secondary Parameters

All members of the HP suite can employ secondary parameters. These embody linear and nonlinear functions of the parameters that a particular program of the HP suite actually manipulates; the values of these secondary parameters can be written to model input files together with the values of primary parameters. See the accompanying PEST_HP manual for details.

One particular member of the HP suite, namely CMAES_HP, can also employ so-called “file parameters”. These are described in the present manual with documentation for that program.

1.7 Observation Penalties

The manual for PEST_HP describes the special significance awarded to observation groups whose names begin with “<@” and “>@”. Membership of observations to this group allows calculation of one-sided residuals. This same functionality is supported by all members of the HP suite described in this manual.

2. JATEST_HP

The role of JATEST_HP is the same as that of the PEST JATEST utility. That is, it undertakes a series of model runs using incrementally varied parameters. This is normally done in order to test differentiability of model outputs with respect to the varied parameter. See part 2 of the PEST manual for details.

While the JATEST utility allows parallelization of model runs, it uses the old parallel PEST protocol for communication between manager and agents. In contrast, JATEST_HP uses the same parallelization protocol as that employed by BEOPEST and PEST_HP. That is, communication between manager and agents is undertaken using TCP/IP. Hence there is no need to prepare a Parallel PEST run management file.

The JATEST_HP manager is run using the command

```
jatest_hp case param nrun outfile /h :nnnn
```

where

<i>case</i>	is the filename base of a PEST control file;
<i>param</i>	is a parameter whose value will be incrementally varied;
<i>nrn</i>	is the number of parameter variations to implement (one less than the number of model runs to undertake);
<i>outfile</i>	is the name of the JATEST_HP output file; and
<i>/nnnn</i>	is a user-selectable TCP/IP port number (use a high number such as 4004 to avoid conflict with other TCP/IP port users).

Each agent is run using the command

```
agent_hp case /h hostname:nnnn
```

where

<i>hostname</i>	is the IP address (version 4) or hostname of the machine on which the JATEST_HP manager is running; and
<i>nnnn</i>	is the TCP/IP port used by the JATEST_HP run manager.

Note the following.

- In the event of model run failure, JATEST_HP does not cease execution. However the outputs of failed model runs are recorded as -1.01D300 on its output file so that they are easily recognized.
- Execution of JATEST_HP can be paused, unpaused and stopped using the PPAUSE, PUNPAUSE, PSTOP and PSTOP utilities supplied with PEST. These commands must be issued in a command line window open to the folder in which the JATEST_HP master is operating.

3. CMAES_HP

3.1 General

CMAES_HP is a HP suite version of the global optimisation utility CMAES_P supplied with the PEST suite. For details of the CMAES algorithm, and of how these details are implemented in CMAES_P, see part 1 of the PEST manual.

Differences between CMAES_HP and CMAES_P are principally as follows.

- In common with other members of the HP suite, CMAES_HP cannot undertake serial model runs; it can only commission parallel model runs.
- CMAES_HP uses TCP/IP for communication between the run manager and run agents. In contrast, CMAES_P uses message files for communication between manager and agents.
- CMAES_HP can employ secondary parameters and file parameters in addition to the parameters that it actually estimates. Use of file parameters opens the possibility of its deployment in optimization of decision variables where parameters are uncertain.

3.2 Running CMAES_HP

The CMAES_HP manager is run using the command

```
cmaes_hp case /r /h :nnnn
```

where

case is the filename base of a PEST control file;
/r is an optional restart switch; and
/nnnn is a user-selectable TCP/IP port number.

There is no limit to the number of agents that can be used in conjunction with a single manager. However it makes no sense to use more agents than the CMAES population size plus the optional number of trial singular value thresholds that it may optionally employ, for this is the number of model runs which comprise a single run package.

Note that there is no need to accept CMAES_HP's default recommendations for population size and number of parents. CMAES_HP does not check for the number of available computing nodes before making these recommendations. If you have access to a greater number of nodes than CMAES_HP's suggested population size, inform CMAES_HP that your population size is accordingly larger. Then set the parent size to about half the population size.

See CMAES_P documentation in part I of the PEST manual for a full description of these, and other, settings.

Each agent is run using the command

```
agent_hp case /h hostname:nnnn
```

where

hostname is the IP address (version 4) or name of the machine on which the CMAES_HP manager is running; and

nnnn is the TCP/IP port used by the CMAES_HP run manager.

3.3 Some CMAES_HP Details

3.3.1 Starting and Stopping

In many respects, operation of CMAES_HP is identical to that of CMAES_P. In common with its CMAES_P counterpart, the CMAES_HP run manager asks the user a series of questions on commencement of its execution. Optionally, each of these questions can be answered by simply pressing the <Enter> key to accept the CMAES_HP default response.

CMAES_HP terminates execution if any of its termination criteria are met. These are supplied by the user in response to CMAES_HP prompts. However CMAES_HP execution can also be terminated at any time by pressing <Ctl-C> while focussed on the run manager's window. Alternatively, execution of CMAES_HP can be temporarily paused and unpaused by typing the "ppause" and "punpause" commands from another command line window that is open to the CMAES_HP manager's working folder. Similarly, execution of CMAES_HP can be terminated by issuing the "pstop" or "pstopst" command from this same window; both of these stopping commands have the same effect on CMAES_HP. The PPAUSE, PUNPAUSE, PSTOP and PSTOPST utilities are supplied with the PEST suite.

If execution of CMAES_HP is terminated by the user, its execution can be resumed by starting it with the "/r" command line switch. If restarted in this way, CMAES_HP does not prompt for user-supplied information; it obtains this information from its restart file. If the CMAES_HP manager is restarted using the "/r" switch, it recommences execution at the start of the iteration in which its previous execution was terminated. It does not read the outcomes of model runs that were previously conducted during this terminated iteration.

On cessation of execution, CMAES_HP does not carry out a final model run based on optimised parameters. At any stage of the CMAES_HP optimisation process (and at the end of the CMAES_HP optimisation process), best parameters achieved up until that point can be found in file *case.par* where *case* is the filename base of the PEST control file on which its run is based. To conduct a single model run based on this optimised parameter set, do the following.

- Use the PARREP utility to write a new PEST control file based on optimised parameters.
- Run PEST or PEST_HP with NOPTMAX set to zero using this file; or run CMAES_HP, informing it (in response to its pertinent prompt) that the maximum number of iterations is zero.

3.3.2 Extra Output Files

CMAES_HP records more files than does CMAES_P. Some of these extra files are the same as those recorded by PEST_HP, as both of these programs share the same run manager.

Suppose that the PEST control file on which a CMAES_HP run is based is named *case.pst*. CMAES_HP records the following files:

- *case.rec*, the run record file;
- *case.ofr*, which tabulates objective functions (total and group) at the end of each iteration;
- *case.par*, which records best parameter values achieved to date;
- *case.par.N*, which records parameter values at the end of iteration N, if parameters were updated during this iteration;

- *case.per*, which lists parameter values associated with failed model runs;
- *case.rme*, which records run agent efficiencies;
- *case.rmr*, which records the communications history between the run manager and each of its run agents.

3.3.3 Forgiving Run Failure

One of the questions that the CMAES_HP manager asks on commencement of its execution is the following:

```
Forgive model run failure? [y/n]: (<Enter> if "y"):
```

If your response to this question is “n” then, should the model fail to run to completion for a particular parameter set, CMAES_HP will attempt to repeat the model run on another agent. If model run failure occurs again, CMAES_HP ceases execution with an appropriate error message. However if your response to the above prompt is “y” (or simply <Enter>), CMAES_HP does not attempt to repeat a failed model run; nor is an error condition incurred. It simply assigns a high objective function value to that run.

3.4 File Parameters

3.4.1 General

CMAES_HP can employ “file parameters” in addition to its normal parameters and secondary parameters. Unlike normal parameters, file parameters are not adjusted through the CMAES_HP optimisation process as it strives to lower the objective function. However random realizations of values for these parameters can be read from external files. These values are transferred, together with those of normal and secondary CMAES_HP parameters, to run agents, and then to model input files through template files. The names of file parameters, and of the template files which feature them, must be provided in the PEST control file read by CMAES_HP, along with the names of normal parameters (and possibly secondary parameters) and the names of template files which cite them.

The purpose of file parameters is to enable CMAES_HP to undertake optimisation under uncertainty along similar lines to that described by Bayer et al (2008) and Bayer et al (2010).

Suppose that a PEST control file names M file parameters. At the beginning of every iteration of the CMAES_HP optimisation process, you can supply values for as many sets of these M parameters as you see fit; each set must include a value for all file parameters. Suppose that you supply values for N such sets; these will normally constitute N random realizations of the set of M file parameters. Then on any occasion that CMAES_HP runs the model during that iteration, it actually runs the model N times. For all of these N model runs it uses the same values for normal and secondary CMAES_HP parameters. However it uses different values for the file parameters – a different set of values on each occasion that it runs the model. These values of file parameters are read from a set of user-nominated parameter value files (hence the name “file parameters”). Each parameter value file in the set may have been written by a program such as RANDPAR or RANDPAR1.

If you wish, N (the number of random file parameter sets, and hence the number of parameter value files that record these parameters) can vary from iteration to iteration of the CMAES_HP optimisation process. Optionally, CMAES_HP can issue a system command through which an external program can be run in order to alter the contents of a file which informs CMAES_HP at the

start of each optimisation iteration of the number of sets of file parameters that it must employ during that iteration, and of the names of the parameter value files from which it must read their values.

3.4.2 Defining File Parameters

The names of file parameters must be provided in the “parameter data” section of a PEST control file. CMAES_HP must be informed of the number of file parameters that reside in this section through a variable named NPARFILE that appears on the fourth line of the PEST control file (in the “control data” section of this file). This is illustrated in figure 3.1. (Note that control variables which govern the use of secondary parameters can also appear on the same line of a PEST control file.)

```
pcf
* control data
restart estimation
10 19 2 10 3 nparfile=9 fileparfile=fpfile.dat
2 3 single point 1 0 0
.
.
```

Figure 3.1 First part of the “control data” section of a PEST control file featuring the NPARFILE and FILEPARFILE control variables.

In the above figure, the value of NPAR (first variable on the fourth line of a PEST control file) is set to 10. Hence the “parameter data” section of the PEST control file cites 10 normal parameters; some of these must be adjustable while others may be tied or fixed. The string “nparfile=9” informs CMAES_HP that, in addition to these 10 normal parameters, the PEST control file features 9 file parameters. (Note that spaces can surround the “=” sign in this string if desired.) The string “fileparfile=filename” informs CMAES_HP of the name of a “file parameter file”. CMAES_HP must read this file in order to obtain the names of the sequence of parameter value files from which the values of file parameters are read. This file parameter file is read at the beginning of each iteration of the CMAES_HP optimisation process; its contents can change from iteration to iteration.

If the value of NPARFILE is provided as M , then the names of M file parameters must be provided in the “parameter data” section of the PEST control file. (Note that there is no limit on the size of M .) A simple example is provided in figure 3.2. Figure 3.3 provides a more complex example.

```
* parameter data
ro1 log factor 1.000000 0.1 100 ro 1 0 1
ro2 log factor 1.000000 0.1 100 ro 1 0 1
ro3 log factor 1.000000 0.1 100 ro 1 0 1
ro4 log factor 1.000000 0.1 100 ro 1 0 1
ro5 log factor 1.000000 0.1 100 ro 1 0 1
ro6 log factor 1.000000 0.1 100 ro 1 0 1
ro7 log factor 1.000000 0.1 100 ro 1 0 1
ro8 log factor 1.000000 0.1 100 ro 1 0 1
ro9 log factor 1.000000 0.1 100 ro 1 0 1
ro10 log factor 1.000000 0.1 100 ro 1 0 1
h1 file_parameter
h2 file_parameter
h3 file_parameter
h4 file_parameter
h5 file_parameter
h6 file_parameter
h7 file_parameter
h8 file_parameter
h9 file_parameter
```

Figure 3.2 The “parameter data” section of a PEST control file which cites 9 file parameters.

```

* parameter data
ro1  log factor 1.000000 0.1 100 ro 1 0 1
ro2  log factor 1.000000 0.1 100 ro 1 0 1
ro3  log factor 1.000000 0.1 100 ro 1 0 1
ro4  log factor 1.000000 0.1 100 ro 1 0 1
ro5  log factor 1.000000 0.1 100 ro 1 0 1
ro6  log factor 1.000000 0.1 100 ro 1 0 1
ro7  log factor 1.000000 0.1 100 ro 1 0 1
ro8  log factor 1.000000 0.1 100 ro 1 0 1
ro9  log factor 1.000000 0.1 100 ro 1 0 1
ro10 log factor 1.000000 0.1 100 ro 1 0 1
h1   log factor 0.25 0.05 100 hhh 1 0 1
h2   tied factor 0.50 0.05 100 hhh 1 0 1
h3   tied factor 1.00 0.05 100 hhh 1 0 1
h4   tied factor 2.00 0.05 100 hhh 1 0 1
h5   tied factor 4.00 0.05 100 hhh 1 0 1
h6   tied factor 8.00 0.05 100 hhh 1 0 1
ff1  file_parameter
h7   tied factor 16.0 0.05 100 hhh 1 0 1
h8   tied factor 32.0 0.05 100 hhh 1 0 1
h9   tied factor 64.0 0.05 100 hhh 1 0 1
h2 h1
h3 h1
ff2  file_parameter
h4 h1
h5 h1
h6 h1
h7 h1
ff3  file_parameter
h8 h1
h9 h1

```

Figure 3.3 The “parameter data” section of a PEST control file which cites 3 file parameters.

Each line in the “parameter data” section of a PEST control file that defines a file parameter must begin with the name of the actual parameter. The string “file_parameter” must follow that. No value is supplied for any file parameter, as the values of file parameters are provided in other files.

As is the case for a normal parameter, the name of a file parameter must be 12 characters or less in length. It must also be unique. Hence it must not have the same name as a normal parameter or a secondary parameter.

The two-word line which defines a file parameter can be placed anywhere within the “parameter data” section of a PEST control file. Where this section is divided into two parts because of the presence of tied parameters, file parameters can be defined in either, or both, of the first and/or second parts of this section.

It is important to note that the variable NPAR defined in the “control data” section of a PEST control file must not account for the presence of file parameters; it must account only for the number of normal parameters. CMAES_HP is informed of the number of file parameters through the NPARFILE keyword.

All file parameters defined in the “parameter data” section of a PEST control file must appear in at least one template file that is cited in the “model input/output” section of the same PEST control file.

Prior information equations appearing in a PEST control file cannot feature file parameters. In contrast, equations which define secondary parameters in the “parameter data” section of a PEST control file can feature one or more file parameters.

3.4.3 Supplying Values for File Parameters

If NPARFILE is set to a value that is greater than zero on the fourth line of a PEST control file, then this same line must provide the name of a file parameter file. This is supplied as the value of the FILEPARFILE variable. Figure 3.4 illustrates the contents of such a file.

```
5 0
11 h11.par
12 h12.par
13 h13.par
14 h14.par
15 h15.par
```

Figure 3.4 A file parameter file.

The first line of a file parameter file must contain two integers. The second integer must be 0 or 1. If the second integer is 0, then the first integer is the number of lines to follow. If the second integer is 1, then the number of lines which follow the header line must be equal to the value of the first integer plus 1. The second integer informs CMAES_HP whether an optional system command is provided on the last line of the file parameter file. If it is set to 0 then a system command is not provided; if it is set to 1 then a system command is provided.

Suppose that the value of the first integer is N . Then each of the following N lines of the file parameter file must contain two items. The first item must be an integer, this being an arbitrary (but unique) identifier for a file parameter set. The second item must be the name of a parameter value file which contains values for that particular set of file parameters.

Note the following.

- The integer identifiers which you ascribe to file parameter sets can be supplied in any order. Their values do not matter. They are used for identification purposes only.
- A parameter value file cited in a file parameter file must contain values for file parameters only. Furthermore, each such file must contain values for all of the file parameters which feature in a PEST control file.

Because the file parameter file is read at the beginning of every iteration of the CMAES_HP optimisation process, it can change from iteration to iteration. The number and names of parameter value files provided during any one iteration of the optimisation process may differ from those provided in any other iteration. Dynamic alteration of the contents of a file parameter file enables construction of dynamic stacks of file parameter sets in the manner described by Bayer et al (2008; 2010). An external program can be used to determine the composition of a file parameter file, and to write this file at the beginning of each CMAES_HP iteration. This program can be run by CMAES_HP using a system command. As stated above, the name of this command can be supplied on the last line of the file parameter file. This is exemplified in figure 3.5.

```
5 1
11 h11.par
12 h12.par
13 h13.par
14 h14.par
15 h15.par
"process_cmaes_outputs.exe"
```

Figure 3.5 A file parameter file which includes a system command as its final line.

Single or double quotes are optional when supplying the system command. If they are omitted, then all characters provided on the last line of the file parameter file comprise the command which CMAES_HP will issue to the operating system at the beginning of each iteration. If they are included, then a single or double quote must comprise the leading character on the final line of the file parameter file; the system command then comprises all characters between the leading quote and a subsequent quote of the same type on that line.

The optional system command is further discussed below.

3.4.4 Calculating the Objective Function

During each iteration of its optimisation process, CMAES_HP calculates an objective function for each member of a population of parameter values (i.e. the values of normal PEST parameters that are included in the NPAR count). The size of the population employed by CMAES_HP can be user-specified. Alternatively, the CMAES_HP default can be accepted. Let us designate the population size as P .

Where file parameters are introduced to the optimisation process, the number of parallel model runs that CMAES_HP undertakes during any iteration of the inversion process is equal to the population size times the number of sets of file parameters that it employs during that iteration; thus the number of model runs is equal to $N \times P$. If no file parameters are defined, then each model run is based on a single set of normal PEST parameters (with secondary parameters possibly defined from these). Where file parameters are employed, each run is repeated N times, with a different set of file parameters used on each occasion. The values of normal (and possibly secondary) PEST parameters do not vary between these N model runs; however the values of file parameters are different from run to run, these having been obtained from pertinent parameter value files in the manner described above.

The objective function associated with any particular set of normal PEST parameters (which generally comprise the decision-variables of an optimisation process) is set to the maximum objective function calculated over all file parameter set realizations. CMAES_HP's task therefore is to adjust the values of its normal parameters (i.e. decision variables) in order to minimize an objective function which, for each set of decision variables, is the maximum over all realizations of file parameters. This is the basis of optimisation under uncertainty.

3.4.5 CMAES_HP Output

CMAES_HP output when using file parameters is very similar to its usual output. The run record file has the same format; however the fact that file parameters are being deployed is recorded in the first section of this file. Likewise, CMAES_HP's screen output with file parameters is similar to that without file parameters. However a noticeable difference to its screen output when using file parameters results from the fact that the number of model runs required per iteration is larger. Hence the record of model run completions that is written to the screen differs from that which would be the case if no file parameters were employed.

In accordance with its normal protocol, CMAES_HP records optimised parameter values in a parameter value file named *case.par*, where *case* is the filename base of the PEST control file. This file makes no mention of file parameters. This is because file parameters are not optimised; they simply express uncertainties associated with certain aspects of a model.

CMAES_HP records one extra output file when using file parameters. This is a “file parameter objective function file”. It is named *case.fpo*, where *case* is the filename base of the PEST control file. Part of a file parameter objective function file is shown in figure 3.6. For this particular example only five realizations of file parameters are used per iteration.

Iteration number: 0			
1	1	11	545.0286
1	2	12	646.0687
1	3	13	845.1376
1	4	14	515.2966
1	5	15	543.8536
Iteration number: 1			
1	1	11	502.3958
1	2	12	492.6022
1	3	13	481.7879
1	4	14	483.4754
1	5	15	502.3958
2	1	11	650.2811
2	2	12	651.8538
2	3	13	658.0548
2	4	14	663.1965
2	5	15	650.2811
3	1	11	528.5220
3	2	12	534.6340
3	3	13	537.7239
3	4	14	539.2051
3	5	15	528.5220
4	1	11	620.1152
etc			

Figure 3.6 The first part of a file parameter objective function file.

The file parameter objective function file records the objective function calculated for all model runs that CMAES_HP commissions during the optimisation process. Apart from section headers which proclaim the iteration number, each line of a file parameter objective function file has four entries. These entries are as follows.

1. The index of the normal CMAES parameter set on which the model run is based. This ranges from 1 to P , where P is the population size. Numbering is sequential, starting at 1.
2. The index of the file parameter set used for the model run. These are also numbered sequentially starting at 1. The highest value recorded for this index during any iteration of the CMAES_HP inversion process is equal to N , where N is the number of parameter value files cited in the file parameter file that CMAES_HP reads at the start of each iteration.
3. The user-supplied integer which identifies each parameter value file cited in the file parameter file.
4. The objective function calculated using the specified normal parameter set in combination with the specified file parameter set.

The file parameter objective function file is available for processing at any time. It is flushed from cached storage at the end of each iteration of the CMAES_HP optimisation process. Hence its latest contents are available for processing by an external program that may be run using the system command that is issued by CMAES_HP at the beginning of the next iteration.

3.4.6 The Optional System Command

As was discussed above, CMAES_HP can optionally read a system command from the last line of a file parameter file. Through this command, the contents of the file parameter file can be altered from iteration to iteration. Hence the number of file parameters used during any iteration of the optimisation-under-uncertainty process can be increased or decreased as appropriate for efficiency and/or reliability of that process. See Bayer et al (2008; 2010) for details. The external program which is run through the system command will presumably obtain the information on which it bases file parameter set alteration from the file parameter objective function file written by CMAES_HP. Note that this system command is issued by the CMAES_HP run manager, but not by any run agents. Processing is thus local to the computer (and working folder) of the run manager.

It is important to understand the sequence of events that occurs during each iteration of a CMAES_HP optimisation process in which CMAES_HP reads a file parameter file (together with parameter value files cited therein) and implements a system command. CMAES_HP actually issues the system command before it reads the file parameter file. The system command which it issues is that which was obtained from the file parameter file which it read during the previous iteration. During the zeroth iteration of the CMAES_HP optimisation process (when it runs the model on the basis of parameter values supplied in the PEST control file), CMAES_HP does not issue a system command at all. The same applies during the first iteration of the optimisation process if CMAES_HP had been asked not to undertake an initial model run based on parameters cited in the PEST control file.

Because CMAES_HP issues the system command prior to reading the file parameter file, the user-supplied program which is run using this command has the opportunity to re-write the file parameter file on the basis of results achieved during previous iterations.

3.4.7 Some Other Implementation Details

Under normal circumstances, PEST_HP and CMAES_HP save parameters that instigate model run failure in a parameter error file. This file has the same filename base as that of the PEST control file; its extension is “.per”. CMAES_HP does not record this file when using file parameters.

As when operating without file parameters, a terminated CMAES_HP run can be re-started using the “/r” switch. When re-started in this way, it re-commences the optimisation process at the beginning of the iteration in which its execution was previously halted. However a re-started CMAES_HP has no recollection of the system command that it may have read from the file parameter file during the previous iteration. You must therefore ensure that the file parameter file which CMAES_HP reads during the first iteration of its re-initiated optimisation process is correct for that iteration.

3.5 Secondary Parameters

As has already been mentioned, CMAES_HP can employ secondary parameters as well as file parameters. Secondary parameters are described in the manual for PEST_HP. Their use in CMAES_HP is identical to their use in PEST_HP, except for the following.

The values of secondary parameters associated with optimised primary parameters are listed at the end of run record files produced by both PEST_HP and CMAES_HP. However CMAES_HP does not record their values if any file parameters are also defined in the PEST control file on which it operates. Omission of the values of secondary parameters under these circumstances makes

allowance for the fact that the value of a secondary parameter may depend on that of a file parameter; if this is the case, then it can have no optimised value. (For efficiency reasons, CMAES_HP does not keep track of whether a secondary parameter does, indeed, depend on a file parameter; it just assumes the worst.)

4. RSI_HP

4.1. Introduction

RSI_HP performs a task similar to that of PESTPP-IES (White, 2018). It iteratively adjusts a set of random parameter realizations which are, presumably, samples of the prior parameter probability distribution, until they constitute samples of the posterior parameter probability distribution. The number of parameters that are featured in each of these realizations can be arbitrarily large (numbering in the tens or hundreds of thousands). However the number of model runs required for realization adjustment is determined by the number of realizations, and not by the number of parameters. Hence adjustment of realizations to conform with calibration constraints is numerically cheap. As presently programmed, RSI_HP allows up to 999,999 parameters to comprise a realization.

Like other members of the HP suite, RSI_HP reads a PEST control file. Within this file it expects to find a section named “rsi”. This section contains variables which control certain aspects of its operation. However a few RSI_HP implementation details are controlled by variables which appear in other sections of the PEST control file.

4.2 Running RSI_HP

RSI_HP parallelizes model runs using the same run management protocols as does PEST_HP. The RSI_HP manager program is run using the command

```
rsi_hp case /h :nnnn
```

where *nnnn* is a port number (a number such as 4004 generally works well) and *case* is the filename base of a PEST control file (which is thus named *case.pst*).

An alternative version of RSI_HP is supplied for use on WINDOWS systems. This is named RSI_HP_MKL. This makes extensive use of the Intel MKL library for matrix and vector operations performed by RSI_HP. Use of this version of RSI_HP is therefore recommended, especially where the number of parameter realizations employed by RSI_HP is high. RSI_HP must regularly invert matrices whose sizes are governed by the number of realizations that it employs. Furthermore, the greater is the number of realizations, the greater is the number of these matrices which it must invert. Most cases of real-world interest require the use of at least 200 realizations. Use of the Intel MKL library can hasten execution speed considerably under these circumstances.

The RSI_HP_MKL version of RSI_HP is run using the command

```
rsi_hp_mkl case /h :nnnn
```

Use of RSI_HP_MKL requires that a dynamic link library file named *libiomp5md.dll* be placed in the folder from which it is run, or in a folder that is cited in the PATH environment variable. If it is not present, then the *rsi_hp_mkl.exe* executable program will not run.

The RSI_HP (or RSI_HP_MKL) manager can be used with as many agents as a user desires. Agents can reside on the same machine as that on which the manager is running; they can also reside on different machines. All agents must work from different folders. Each agent is run using the command

```
agent_hp case /h hostname:nnnn
```

where *hostname* in the above command must be replaced by the name of the computer on which RSI_HP is running or by the (version 4) IP address of this machine, and *nnnn* must be replaced by the RSI_HP manager's port number.

Each instance of AGENT_HP must read the same PEST control file as that which the RSI_HP manager reads. It must have access to all template and instruction files that are cited in this control file. The local instance of the model which it runs must have access to all of the files that it requires.

Because RSI_HP employs the same run manager as does PEST_HP, JACTEST_HP and CMAES_HP, it shares the same run management control variables, and writes the same run management output files as these programs. See sections 1.4 and 1.5 of this manual.

In common with other members of the HP suite (and with PEST itself), execution of RSI_HP can be paused, unpaused and stopped using the PPAUSE, PUNPAUSE, PSTOP and PSTOPST utilities supplied with the PEST suite. Type the names of these utilities in a command line window open to the RSI_HP manager's folder. Alternatively, type <Ctl-C> in the RSI_HP manager's window to instigate immediate cessation of its execution.

As presently programmed, RSI_HP cannot be restarted at the same place as its execution was previously interrupted. However, an effective restart is achieved by providing it with a set of parameter realizations achieved during a previous RSI_HP run as its initial ensemble.

4.3 Theory

4.3.1 Background

In some respects, the algorithm employed by RSI_HP resembles that described by Raanes et al (2019) in that parameter adjustment takes place in realization space rather than in parameter space. However the algorithm employed by RSI_HP has some marked differences from the Raanes et al algorithm. These differences can lengthen the time required for calculation of realization upgrades, especially when the number of realizations is large. For this reason the number of realizations employed by RSI_HP should be no greater than about a thousand in order to preclude excessive adjustment times. However these algorithmic differences can also reduce the number of iterations for which parameter adjustment is required, and hence the number of model runs required by RSI_HP to adjust realizations of an ensemble in order for them to support a good fit between model outputs and a calibration dataset.

The algorithm employed by RSI_HP also bears some resemblance to the "random mixing" algorithm described by Bárdossy and Hörning (2016) and Haese et al (2017).

The following explanation of the RSI_HP algorithm is simple. Hopefully it provides enough details for a user to understand its underlying principles, and the variables that control its operation.

4.3.2 Basic Equations

We employ similar nomenclature as that used by Doherty (2015).

Let the relationship between the m parameters $\mathbf{k}$ employed by a model and the n observations $\mathbf{h}$ comprising a calibration dataset be represented by the following matrix equation.

$$\mathbf{h} = \mathbf{Z}\mathbf{k} + \boldsymbol{\varepsilon} \quad (4.1)$$

The vector $\boldsymbol{\varepsilon}$ holds measurement noise.

Equation 4.1 represents the action of a model on its parameters $\mathbf{k}$ as multiplication by a matrix $\mathbf{Z}$. This relationship is therefore linear. For simplicity, offsets are ignored in the above linear equation; hence when $\mathbf{k}$ is zero, model outputs corresponding to measurements of the calibration dataset are also zero.

Suppose that we generate r realizations of parameters. Let us refer to these realizations as $\mathbf{k}_1, \mathbf{k}_2, \dots, \mathbf{k}_r$. At the same time, we also generate r realizations of observations by adding r realizations of measurement noise to the observation dataset $\mathbf{h}$. We refer to these observation realizations as $\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_r$. Let us now select the j 'th member of each of these sets of realizations and formulate an inverse problem defined by the following equation.

$$\mathbf{h}_j = \mathbf{X}_j \mathbf{p}_j + \boldsymbol{\varepsilon}_j \quad (4.2)$$

In equation 4.2, $\mathbf{X}_j$ is an $n \times (r-1)$ matrix whose i 'th column is comprised of the vector $\mathbf{o}_i - \mathbf{o}_j$. $\mathbf{o}_i$ is comprised of model outputs that correspond to members of the calibration dataset calculated using parameter set $\mathbf{k}_i$ while $\mathbf{o}_j$ is comprised of these same model outputs calculated using parameter set $\mathbf{k}_j$. $\mathbf{X}_j$ thus forms a kind of "realization Jacobian matrix". The inverse problem posed by equation 4.2 seeks a set of $r-1$ numbers which comprise elements of the vector $\mathbf{p}_j$. These are factors by which each realization difference $\mathbf{k}_i - \mathbf{k}_j$ must be multiplied before being added to realization $\mathbf{k}_j$ in order for the new $\mathbf{k}_j$ realization to fit the calibration dataset $\mathbf{h}_j$ as well as possible in the weighted least squares sense. The solution to this inverse problem is obtained as

$$\mathbf{p}_j = (\mathbf{X}_j^t \mathbf{Q} \mathbf{X}_j)^{-1} \mathbf{Q} \mathbf{X}_j^t \mathbf{h}_j \quad (4.3)$$

where $\mathbf{Q}$ is a user-supplied weight matrix, ideally calculated as

$$\mathbf{Q} = \alpha \mathbf{C}^{-1}(\boldsymbol{\varepsilon}) \quad (4.4)$$

In equation 4.4, $\mathbf{C}(\boldsymbol{\varepsilon})$ is the covariance matrix of measurement noise and α is an arbitrary constant.

In posing the inverse problem of model calibration in this way, each realization can be thought of as a kind of "super parameter" comprised of a linear combination of real model parameters. Each of these super parameters is modified by addition of differences between it and other super parameters until the fit between model outputs and the corresponding noise-enhanced calibration dataset is minimized in the weighted least squares sense. Because all of these adjusted realizations fit the calibration dataset while preserving as much as possible of their original randomness, they can be considered to be samples of the posterior parameter probability distribution.

Because the relationship between parameters and model outputs is nonlinear, equation 4.3 must be solved iteratively. During each iteration of the super parameter inversion process, the $\mathbf{X}$ matrix is reformulated for all parameter realizations, and then solved r times to upgrade each $\mathbf{k}_j$. This r -fold solution process is then repeated as many times as is required for model-to-measurement fits to be considered good enough for all r members of the ensemble of parameter realizations to be considered as samples of the posterior parameter probability distribution.

Conceptually, during any iteration of the inversion process there is no need to postpone the updating of $\mathbf{X}_j$ until all r parameter realizations have been updated, for it can be updated incrementally as each $\mathbf{k}_j$ is updated. However it is convenient to postpone its updating until the beginning of each new iteration, as this assists the following implementation details:

- the choice of an (iteration-specific) Marquardt lambda;
- application of “diversity regularization”; and
- parallelization of model runs.

All of these matters are discussed below.

RSI_HP solves equation 4.3 by subjecting the $\mathbf{X}_j^t \mathbf{Q} \mathbf{X}_j$ matrix to singular value decomposition. Hence RSI_HP can tolerate singularity of this matrix arising from ill-posedness of the inverse problem. Truncation of singular values is governed by the EIGTHRESH variable that is provided in the “singular value decomposition” section of a PEST control file. (All other variables supplied in this section are ignored.) If a PEST control file does not contain a “singular value decomposition” section, then RSI_HP assigns a default value of 5×10^{-7} to EIGTHRESH.

4.4 Some Algorithmic Details

In this subsection, capitalized words refer to PEST and RSI_HP control variables. All of the latter are discussed in greater detail in a following subsection.

4.4.1 Parameter Transformations

In the “parameter data” section of a PEST control file, the PARTRANS variable associated with each parameter is awarded a value of “none”, “log”, “fixed” or “tied”. Attributes of “fixed” and “tied” perform the same role in RSI_HP as they do in PEST. A parameter whose PARTRANS attribute is “fixed” retains the value ascribed to it in the “parameter data” section of the PEST control file throughout the inversion process. If a user-supplied parameter realization provides a value other than this for a fixed parameter, the value in the realization is replaced by the value of this parameter supplied in the PEST control file.

At all stages of the inversion process the value of a tied parameter maintains the same ratio with its parent parameter as that provided in the “parameter data” section of a PEST control file. The values of tied parameters provided in user-supplied parameter realizations are ignored.

Where a parameter is designated as log-transformed in a PEST control file, the value adjusted by RSI_HP is actually the log (to base 10) of that parameter. Log-transformation of parameters has the same affect in the inversion problem solved by RSI_HP as it does in that solved by PEST; that is, it can promote greater linearity of the inverse problem and thereby hasten convergence of its solution. Hence if a modeller feels that a parameter should be log-transformed when using PEST, then he/she should also log-transform it when using RSI_HP.

It is important to note, however that, regardless of the transformation status of any parameter, parameter values provided to RSI_HP in user-supplied ensembles should be their actual values and not their log-transformed values. Presumably, when generating random realizations of log-transformed parameters in order to populate an ensemble, probability distributions will have been ascribed to the logs of log-transformed parameters rather than to their native values; however this is the user’s responsibility.

4.4.2 The Marquardt Lambda

As is usual practice in solving a nonlinear inverse problem, equation 4.3 is supplemented by inclusion of the Marquardt lambda λ . It then becomes

$$\mathbf{p}_j = (\mathbf{X}_j^t \mathbf{Q} \mathbf{X}_j + \lambda \mathbf{I})^{-1} \mathbf{Q} \mathbf{X}_j^t \mathbf{h}_j \quad (4.5)$$

At the start of the ensemble adjustment process, an initial Marquardt lambda is chosen by RSI_HP. During each iteration of the inversion processes it then attempts to find the best lambda to employ during that iteration by testing lambdas which are varied upwards and downwards from this initial value, or from the best lambda that it found during the previous iteration. Before it solves equation 4.5 for all realizations comprising an ensemble, RSI_HP selects NUMREALLAM realizations in order to perform testing of candidate lambdas. NUMLAMTEST candidate Marquardt lambdas are chosen for testing. One of these is the best lambda from the previous realization (or the initial Marquardt lambda), while the others are varied upwards and downwards from this. Thus a total of $\text{NUMLAMTEST} \times \text{NUMREALLAM}$ model runs are committed to Marquardt lambda testing during each iteration of the ensemble inversion process. On the basis of outcomes of these trial realization upgrades, RSI_HP chooses the most appropriate lambda to employ in equation 4.5 for upgrading of all remaining realizations.

The user-supplied value for the REALLAMSELECT control variable dictates whether the NUMREALLAM lambda-testing realizations are selected randomly, or whether the NUMLAMTEST worst performing realizations are selected for this purpose. By giving the latter realizations special attention (by applying dedicated Marquardt lambdas to their adjustment), the chances of their improvement are increased. Nevertheless, experience demonstrates that random realization selection for the purpose of Marquardt lambda testing often works better.

4.4.3 Supplying Realizations to RSI_HP

It is the user's responsibility to provide initial parameter realizations to RSI_HP. Optionally realization observations can be supplied as well. These must be supplied in so-called "JCB files". These are binary files that are not unlike JCO files that are used by PEST to hold a Jacobian matrix. Parameter realizations can be generated using the PEST RANDPAR3 utility, while observation realizations can be generated using the RANDOBS utility. Each of these utilities stores the realizations which it generates in a JCB file. They also endow each realization with a name. This name is either "base" or an index starting with 1. These names are used to reference realizations in RSI_HP screen and file output.

Observation realisations can be generated internally by RSI_HP. This is described in the next subsection. The pertinent RSI_HP control variable is READ_OBS_ENSEMBLE.

Optionally, RSI_HP can include parameter values recorded in the PEST control file as one of its parameter realizations, and observation values recorded in the PEST control file as one of its observation realizations. Selection of these "base" realizations is enabled by setting the INCLUDE_BASE control variable to 1. It is generally a good idea to do this. However be careful not to include the base realizations twice, as both the RANDPAR3 and RANDOBS utilities can also optionally include parameter and observation values from a PEST control file in their JCB files along with the random parameter realizations and random-noise-supplemented observation realizations which they generate; the name "base" is also associated with these realizations by the respective utilities.

Through the LOSE_IF_NO_IMP and LOSE_PHI_QUANTILE control variables, RSI_HP can be instructed to abandon realizations that undergo no improvement through the ensemble adjustment process. This may enhance RSI_HP's ability to improve other realizations because the "realization Jacobian matrix" on which basis these improvements are calculated attains greater accuracy with the omission of recalcitrant parameter sets. However realization diversity may be lost. So too are the number of realization-based super parameters that are combined to fit the calibration dataset; the fit that is achieved with that dataset may therefore suffer.

As well as reading parameter and observation realizations at the commencement of the ensemble adjustment process, RSI_HP can also read so-called "secondary realizations". It may also be asked to read so-called "supplementary realizations" later in the ensemble adjustment process. These matters are discussed in following subsections.

4.4.4 Using RSI_HP to Generate Observation Realisations

If the READ_OBS_ENSEMBLE RSI_HP control variable is set to 1, then RSI_HP reads observation realisations (except for the "base" realisation) from a JCB file in the manner described above.

As presently programmed, RSI_HP provides two other alternatives for generation of observation realisations.

The first of these two alternatives is activated by setting READ_OBS_ENSEMBLE to 0. In this case no noise is added to observations that are provided in the PEST control file that is read by RSI_HP. The value of a particular observation is therefore the same for all realisations, with no noise added to it.

If READ_OBS_ENSEMBLE is set to -1, then RSI_HP generates observation noise itself (except for the base observation realisation); observation realisations are therefore not read from an external file. Two noise-generation options are possible. If measurement noise standard deviations are provided in an external observation data file (see below), then these are used for generation of observation noise. If they are not provided, then the weight associated with each observation in the "observation data" section of the PEST control file is assumed to be the inverse of the standard deviation of observation noise (unless the observation weight is zero, in which case no noise is added to the respective observation).

4.4.5 Supplying Observations Externally

If instructed to do so, RSI_HP can read observation data from an external file. Its protocol for doing this is similar to that employed by the PEST++ suite. If an external file is provided, this file must contain observation standard deviations as well as observation names, values, weights and observation group names.

If the word "EXTERNAL" follows the "* observation data" section header, then RSI_HP is made aware of the fact that it must read observation data from an external file. Only one such file is allowed. This file must contain all observations that feature in the current inverse problem.

The name of the external file must follow on the ensuing line of the PEST control file. This is illustrated in Figure 4.1.


```
* observation data external
  observ.dat
```

Figure 4.1. “Observation data” section of a PEST control file for which observation data is supplied externally.

Figure 4.2 shows the first part of an external observation data file.

OBSNME	OBSVAL	WEIGHT	OBSNME	STANDARD	DEVIATION
well101_1		5.747240	10.0	heads	0.1
well102_1		5.696160	10.0	heads	0.1
well103_1		5.491990	10.0	heads	0.1
etc					

Figure 4.2 First part of an external observation data file.

The first line of the external observation data file must contain the header that appears in the first line of Figure 4.2. Each ensuing line must have five entries. Observation noise standard deviations must be provided in the final column of this file. Columns can be space or comma delimited.

4.4.6 Prior Information

A PEST control file that is read by RSI_HP can contain prior information.

In some ways, RSI treats prior information just like other observations. However there are some important differences. The principle difference is that user-supplied observation ensembles do not need to include observed values for prior information equations. RSI_HP reads observed values for prior information equations from the PEST control file. It transfers these internally to all observation realisations; in doing so, it adds no random noise to them.

The primary use of prior information equations is for introduction of regularisation to the RSI_HP inversion process. This is described below.

4.4.7 Covariance Matrices

RSI can read a PEST input dataset in which covariance matrices are associated with some observation groups. However observation groups to which covariance matrices are assigned must contain only prior information. RSI does not allow a covariance matrix to be associated with an observation group which contains measured data.

4.4.8 Adjustment Constraints

As realizations are adjusted, relative and factor change limits are imposed on individual parameters whose names are provided in the PEST control file on which the current ensemble inversion process undertaken by RSI_HP is based. These change limits are provided as the RSI_RELPARMAX and RSI_FACPARMAX control variables in the “rsi” section of a PEST control file. These have the same roles as the RELPARMAX and FAXPARMAX control variables that are supplied in the “control data” section of a PEST control file. However they are replicated in the “rsi” section of the PEST control file so that a user does not forget to provide suitable values for them. Optimal values for these variables may be different when used by RSI_HP than when used by PEST or PEST_HP. Limited experience to date suggests that optimal settings for RSI_RELPARMAX and RSI_FACPARMAX are 100.0 or greater. When imposing these constraints, the upgrade vector that is added to any realization is shortened if necessary to prevent any parameter from breaching its specified change limit.

Alternatively, or as well, a limit can be directly imposed on changes to realizations. This is done using the SUPPARMAX control variable which also resides in the “rsi” section of a PEST control file. For any

proposed realization change vector, RSI_HP evaluates the maximum change that any parameter which is featured in this vector will undergo, taking its transformation status into account. It will be recalled from the above discussion that RSI_HP's internal representation of a parameter depends on the parameter's PARTRANS setting. For parameters that are log transformed, the parameter is represented internally in RSI_HP as its log (to base 10); in evaluating SUPPARMAX-governed super-parameter constraints, the change to a parameter is calculated on that basis. If a computed realization update vector decrees that any parameter undergo a change that exceeds the user-supplied value of SUPPARMAX, this vector is shortened to a length that forestalls this occurrence. A value of 100.0 is presently recommended for SUPPARMAX.

It is apparent from the above discussion that presently suggested values for RSI_HP parameter change limits are quite large. This is an outcome of formulating the inverse problem in realization space rather than in parameter space. Parameter normalization is built into the realization-based inverse problem formulation.

4.4.9 Parameter Bounds

Tied parameters do not respect bounds ascribed to them in a PEST control file as their parent parameters are varied by RSP_HP; they simply maintain a constant ratio with their parent parameters, regardless of where this ratio takes them.

For other parameters, the way that bounds are enforced (or not enforced) depends on the value supplied for the IBOUNDOPT control variable in the "rsi" section of a PEST control file. Depending on the value supplied for this variable, bounds can be ignored, respected, or accommodated through "bounds transformation".

If IBOUNDOPT is set to 0, the bounds ascribed to all parameters are ignored during the realization adjustment process. (Note however that a log-transformed parameter will never become negative due to the fact that RSI_HP represents this parameter internally as its log). If IBOUNDOPT is set to 1, then the values of parameters that come into contact with their bounds are clipped at these bounds. If IBOUNDOPT is set to 2, then realization upgrade vectors are shortened to a length that prevents any parameter from violating its bound. A disadvantage of setting IBOUNDOPT to 2 is that this can limit realization diversity, and may even prevent the attainment of a good fit between model outputs and the calibration dataset.

If IBOUNDOPT is set to 3, RSI_HP transforms each parameter internally to represent it as

$$k_t = \ln \left(\frac{(k - k_{lb})}{(k_{ub} - k)} \right) \quad (4.6)$$

In equation 4.6, k is the value of a parameter, k_{lb} is the parameter's lower bound as recorded in the PEST control file, while k_{ub} is its upper bound. k_t , the transformed parameter, has bounds of plus and minus infinity. This transformation eliminates the need for RSI_HP to place bounds on parameters as realizations are updated. However it does cause some distortion of parameter correlations embodied in parameter realizations as the latter are combined with each other to form new realizations that lower the objective function. Limited experience to date suggests that this distortion compromises the performance of RSI_HP.

4.4.10 Objective Functions

On each occasion that RSI_HP runs the model, it associates an objective function with a parameter realization. It calculates this objective function in the same way that PEST calculates an objective function, i.e. as the sum of squared, weighted residuals. Weights are supplied in the “observation data” section of a PEST control file.

As has already been explained, “observations” that are employed for calculation of residuals are realization-specific. For each parameter realization there is a corresponding observation realization. Initial parameter realizations are read from a user-prepared parameter ensemble file. Initial observation realizations are read from a user-prepared observation ensemble file. Each of these ensembles resides in a JCB file. Alternatively, as described above, observation realisations can be generated by RSI_HP itself. Each parameter realization is linked to a specific observation realization; linkages between realizations in these two files are sequential (realization names provided in these two files are ignored). Note, however, that if the INCLUDE_BASE control variable is supplied as 1, then the first parameter realization employed by RSI_HP is comprised of initial parameter values supplied in the “parameter data” section of the PEST control file. Similarly, the first observation realization is comprised of observation values listed in the “observation data” section of this same PEST control file. Each of these realizations is endowed with the name “base”. Obviously, parameter realizations change through the inversion process, for it is the purpose of RSI_HP to improve them. Observation realizations do not change, however.

By running the model, RSI_HP is able to associate a vector of model-to-measurement residuals with each parameter/observation realization pair. Each element of this residuals vector is the difference between a measurement-noise-enhanced observation and its corresponding model output. The objective function is formed by multiplying each residual by the respective observation weight, squaring this product, and then summing these products over all observations comprising a realization. The objective function associated with a parameter/observation realization pair is re-calculated whenever a parameter realization is updated.

RSI_HP associates another objective function with each parameter realization. In keeping with parlance employed by PESTPP-IES, this objective function is referred to as the “actual objective function”. In formulating this objective function, each residual is calculated as the difference between the value of an observation recorded in the PEST control file, and the value of the corresponding model output calculated using the pertinent parameter realization. The “actual objective function” is calculated only for user interest. It forms no part of the RSI_HP inversion algorithm.

When RSI_HP is run in “regularisation” mode, the objective function that is associated with each realisation is subdivided into a measurement objective function and a regularisation objective function (just as it is when PEST is run in “regularisation” mode). This is described below.

4.4.11 Upgrade Rejection

During each iteration of the inversion process, RSI_HP upgrades each of the realizations that comprise the parameter ensemble. After doing this, it runs the model once for each realization in order to evaluate model outputs that correspond to members of the calibration dataset. It calculates model-to-measurement residuals using the noise-enhanced observation realization that corresponds to each parameter realization. It then associates an objective function with each updated realization.

Ideally, the objective function that is associated with each realization should decrease during every iteration of the inversion process. If it does not decrease for a particular realization then, depending on the setting of the REJECT_UPDATE control variable, the updated parameter realization can be rejected and the non-updated realization retained. Suppose that REJECT_UPDATE is set to 1 and that the REJECT_THRESHOLD control variable is set to x (a number greater than 1.0). Then an updated realization will be rejected if the objective function corresponding to this updated realization is greater than the previous objective function by more than a factor of x . A value of 2.0 or less is often suitable for REJECT_THRESHOLD.

If the model fails to run when provided with parameters comprising an updated realization, then an objective function of infinity is associated with that realization. If REJECT_UPDATE is set to 1, the realization is automatically rejected. If REJECT_UPDATE is not set to 1, the realization is abandoned; the inversion process then carries on without that realization.

4.4.12 Realization Abandonment

Any realization which precipitates model failure during the first iteration of the RSI_HP inversion process (when RSI_HP employs parameter realizations that it reads from the user-supplied parameter ensemble file) is abandoned. If model run failure occurs during a later iteration, the guilty realization is abandoned unless REJECT_UPDATE is set to 1, in which case the realization update is rejected and the pre-updated realization is retained.

The “rsi” section of a PEST control file includes variables named LOSE_IF_NO_IMP and LOSE_PHI_QUANTILE. Suppose that LOSE_IF_NO_IMP is set to N . Then if the objective function associated with a particular realization undergoes no improvement for N successive iterations, that realization is abandoned; however it is only abandoned if the objective function that is currently associated with that realization is in the upper quantile of objective functions that is set by LOSE_PHI_QUANTILE. For example if LOSE_PHI_QUANTILE is set to 0.9, then the objective function must be in the highest 10 percent of realization objective functions for the realization to be abandoned. With abandonment of a realization, the number of realizations comprising the ensemble decreases by 1.

4.4.13 Diversity Regularization – a Simple Option

Sometimes, ensemble-based history matching can precipitate “ensemble collapse” wherein updated parameter realizations do not retain sufficient diversity to guarantee the integrity of post-calibration predictive uncertainty analysis. In order to forestall this occurrence, RSI_HP supports the optional deployment of “diversity regularization” in its ensemble adjustment process. This occurs if the REG_DIVERSE control variable is set to 1 or 2. The first of these options is discussed in the present subsection. The other option is available if RSI_HP is run in “regularisation” mode. This is discussed below.

If REG_DIVERSE is set to 1, RSI_HP calculates the average distance in parameter space between initial parameter realizations at the commencement of its inversion process. The squared distance between realization j and realization l is calculated using the formula:

$$d_{jl}^2 = \sum_{i=1}^m \left[\frac{(k_{ji} - k_{li})}{|b_i|} \right]^2 \quad (4.7)$$

Recall that m is the number of adjustable parameters that comprise a parameter realization. Meanwhile b_i is the difference between the upper and lower bounds of parameter i . The transformation state of parameters (i.e. whether or not they are log-transformed) is taken into account in evaluating equation 4.7. The average inter-realization distance is obtained by averaging d_{ji}^2 over all pairs of realizations, and then taking the square root of this average.

Once it has calculated the average initial inter-realization separation, RSI_HP supplements the observation dataset with another set of “observations”. These state that the separation between each adjusted realization and every other adjusted realization remains equal to the average initial inter-realization separation. Each such observation must be assigned a weight. RSI_HP calculates this weight itself; it alters this weight on every occasion that a realization is updated.

RSI_HP calculates weights assigned to observations which implement diversity regularization using a control variable named REGDIVFRAC. Suppose that REGDIVFRAC is set to 0.10. Then RSI_HP ascribes weights to diversity observations in a way which guarantees that corresponding diversity residuals make a contribution to the total objective function that is equal to 0.1 times the current value of the “measurement objective function”. (The measurement objective function records model-to-measurement misfit.)

Experience to date suggests that while diversity regularization does, indeed, constitute a successful mechanism for increasing realization diversity, its use may reduce the speed with which RSI_HP is able to attain a good fit with a calibration dataset.

4.4.14 Secondary Parameter Realizations

RSI_HP provides a mechanism for potentially adding parameterization detail to the “primary realizations” which constitute its main parameter ensemble. However it will only add this detail to a particular realization if its addition allows that realization to attain a better fit with the calibration dataset than could be achieved without it.

As implied above, we use the term “primary parameter realizations” in this subsection to describe the set of realizations that RSI_HP reads upon commencement of its execution, and then adjusts as its execution progresses. RSI_HP can also read a set of so-called “secondary parameter realizations” at the commencement of its execution that collectively comprise a “secondary parameter ensemble”. These realizations contain modifications that can be potentially imposed on primary parameter realizations.

Modification of a primary parameter realization by a secondary parameter realization takes place through multiplication of individual parameter elements comprising the primary realization by corresponding parameter elements comprising the secondary realization where these parameter elements are log-transformed; it takes place by addition of parameter elements comprising a secondary realization to corresponding parameter elements comprising a primary realization for parameters that are not log-transformed. (The logs of primary and secondary parameter values are added in the former case, for this is equivalent to multiplication in non-log space.) Hence realizations of secondary parameters that are provided to RSI_HP by a user should be centred on 1.0 for log-transformed parameters, and on 0.0 for non-log-transformed parameters. (Such realizations can be generated using the RANDPAR4 utility.) Tied and fixed parameters are not altered by secondary parameters.

The use of secondary parameter realizations entails introduction to the RSI_HP inversion process of another set of p_j parameters for use in equations 4.2 and 4.5. These parameters are the factors by which model parameters comprising secondary realizations (or their logs) are multiplied before being added to the model parameters (or their logs) comprising a particular primary realization. Values for this set of “modification parameters” are specific to each primary realization, and are estimated as part of the RSI_HP inversion process together with factors that are applied to primary realizations. However, despite the fact that these secondary realization adjustment factors are primary-realization-specific, derivatives of model outputs with respect to these adjustment factors are calculated just once during each RSI_HP realization using the currently best fitting primary parameter realization as their base. These same derivatives are then used in estimating values for secondary realization adjustment factors that pertain to all other primary parameter realizations, as the latter are also modified by secondary realizations and combined with other primary realizations to obtain an updated primary parameter ensemble.

In summary, during each iteration of the RSI_HP inversion process, each primary parameter realization is modified through the addition/multiplication of differences between it and all other primary parameter realizations. However where secondary realizations are employed, each primary parameter realization can also be modified through the addition/multiplication of all secondary parameter realizations. Factors that are applied to both primary and secondary realizations are calculated by RSI_HP. These factors are specific to each primary realization.

Secondary parameter realizations comprising the secondary parameter ensemble are read by RSI_HP at the beginning of its execution, just after it reads the primary parameter ensemble and the observation ensemble. However secondary parameter realizations are employed for primary ensemble modification only at user-specified iterations. During those iterations, RSI_HP must undertake as many model runs as there are primary realizations plus as many model runs as there are secondary realizations; the latter set of runs are used to calculate derivatives of model outputs with respect to secondary realization adjustment factors.

Generally, the secondary parameter ensemble should possess fewer realizations than the primary parameter ensemble. As stated above, each member of the former is able to modify all members of the latter. Secondary realizations may therefore include geobodies of various shapes. The RSI_HP inversion process is thus given the opportunity to incorporate one or more of these shapes into thereby modified primary realizations if this supports a better fit with the calibration dataset.

Note that secondary realizations cannot be employed in an RSI_HP realization adjustment process that also employs supplementary realizations. Only one or the other of these (or neither of them) can be employed at the same time.

4.4.15 Supplementary Parameter Realizations

As stated above, RSI_HP reads user-supplied parameter realizations comprising a parameter ensemble immediately on commencement of its execution. Optionally, it can read “supplementary parameter realizations” later in the inversion process. This does not increase the number of realizations that it adjusts, nor the dimensionality of realization space in which it operates. Supplementary realizations are used for one of two purposes:

- to replace some of the parameters featured in current realizations;

- to modify some parameters in currently employed realizations through addition or multiplication.

Through this means, supplementary parameter realizations can introduce additional complexity to existing parameter realizations after the latter have been at least somewhat adjusted in order to more closely conform with calibration constraints.

Suppose that the USESUPINT control variable in the “rsi” section of a PEST control file is set to 3. Then RSI_HP will read supplementary parameter realizations from a user-nominated JCB file on every third iteration of the inversion process. That is, it will read supplementary realizations on the 3th, 6th, 9th etc iterations of the inversion process. However it will not read supplementary parameter realizations after the iteration whose number is supplied as the MAXSUPITN control variable. (Setting USESUPINT to zero disables supplementary realization functionality.)

Supplementary parameter realizations must be provided in a JCB file; this must not be the same JCB file as that in which initial (i.e. primary) parameter realizations are provided. The supplementary parameter ensemble file continues to be read throughout the ensemble inversion process. A user must therefore ensure that it contains enough parameter realizations for NREAL of these realizations to be read during each iteration in which supplementary realizations are employed. NREAL is the number of primary parameter realizations that are used by RSI_HP; this is the number of realizations that are modified by NREAL supplementary realizations.

As it reads NREAL supplementary parameter realizations, RSI_HP pairs each one of them with an existing primary parameter realization; pairing is sequential. Supplementary parameter values alter corresponding primary parameter values featured in a paired primary realization only if absolute values of the former are less than a threshold supplied as the value of the SUPTHRESHVAL control variable. Thus if a parameter’s value in a supplementary realization is above this threshold, it does not alter the value of the corresponding parameter in the primary realization to which it is paired. If the SUPREPLACE control variable is set to 0, then the supplementary parameter value is added to the existing primary parameter value if this parameter has a PARTRANS setting of “none”; however if its PARTRANS setting is “log”, then the supplementary parameter value multiplies the existing primary parameter value. Alternatively, if SUPREPLACE is set to 1, then the supplementary parameter value replaces the primary parameter value.

Optionally, parameter values comprising supplementary realizations are all multiplied by a factor before being used in the above manner. This factor is supplied as the value of the SUPFRAC control variable. However the SUPTHRESHVAL threshold criterion is enforced before this factor is applied.

During those iterations in which supplementary realizations are read by RSI_HP, existing, primary parameter realizations undergo no other manipulation. In particular, no realization improvement is attempted using equation 4.5; this is delayed until the ensuing iteration. Hence it is likely that the objective function value associated with each primary parameter realization will rise during each iteration in which supplementary parameter realizations are read. (Objective functions are reported at the start of the next iteration.) To accommodate this, the objective function threshold which precipitates realization rejection can be temporarily increased for this iteration. This is accomplished by supplying an appropriate value for the TEMP_REJECT_THRESHOLD control variable. As has already been discussed, for normal iterations REJECT_THRESHOLD is the factor by which the objective

function corresponding to a particular realization is multiplied in order to determine whether a realization update is rejected or not. If the new objective function is greater than the existing objective function times REJECT_THRESHOLD, the updated realization is rejected if REJECT_UPDATE is set to 1. TEMP_REJECT_THRESHOLD replaces REJECT_THRESHOLD for iterations in which supplementary realizations are read.

Note that supplementary realizations cannot be employed in an RSI_HP ensemble adjustment process that also employs secondary realizations. Only one or the other of these (or neither of them) can be employed at the same time.

4.4.16 Multiple Data Assimilation

RSI_HP has a mode of operation which is not unlike “multiple data assimilation” functionality that is available with ensemble methods. However there are some important differences.

Multiple data assimilation is governed by the NMDA control variable. If this variable is set to a number greater than 0, then the READ_OBS_ENSEMBLE control variable must be set to -1; RSI_HP therefore generates observation noise itself rather than reading it from an external file.

Suppose that NMDA is set to 5. Then on the first iteration of the inversion process RSI_HP generates random noise with a variance (i.e. square of standard deviation) that is 5 times greater than that which is warranted on the basis of weights or standard deviations supplied in the “observation data” section of the PEST control file (or from an external observation data file that is read in its stead). Then, on the second iteration of the inversion process it regenerates random noise; in this case it employs a noise variance which is 4 times greater than warranted. This noise regeneration process continues until the fifth iteration, at which point correct noise variances are employed. No new realisations of measurement noise are generated during subsequent iterations of the inversion process.

Note that a significant difference between the “multiple data assimilation” scheme as undertaken by RSI_HP and that undertaken by an ensemble Kalman filter or smoother, is that the inverse of the sum of noise variances over NMDA iterations is not constrained to sum to 1.0.

4.4.17 Termination Criteria

There is no need for a user to patiently wait for RSI_HP to finish execution of its own accord. Its execution can be brought to an abrupt halt by pressing <Ctl-C> in the manager window at any time. As is explained in the following section, all of RSI_HP’s output files are written or extended during every iteration of the ensemble inversion process. Hence outcomes of this process are available for inspection at any time (even while RSI_HP is running).

The NOPTMAX termination control variable used by PEST is also used by RSI_HP. As is documented in part 1 of the PEST manual, this control variable is supplied on the 9th line of a PEST control file (in the “control data” section of this file). A PEST or RSI_HP inversion process automatically terminates after NOPTMAX iterations have elapsed. If NOPTMAX is set to 0, then RSI_HP runs the model once for each realization, records objective function statistics, stores realizations and corresponding model outputs in pertinent JCB files (see below), and then ceases execution. If NOPTMAX is set to 1, then realizations are upgraded once; RSI_HP then ceases execution during iteration 2 after it has run the model using all updated realizations, recorded results and written pertinent output files. Unlike PEST, RSI_HP does not accept negative NOPTMAX values.

Termination of RSI_HP execution is also controlled by operation of the PHIREDTYPE, RELPHIRED and NPHIRED control variables supplied in the “rsi” section of a PEST control file. RELPHIRED and NPHIRED are applied to either the average, minimum or maximum objective function calculated over all ensemble realizations; the choice is made using the PHIREDTYPE control variable. If the selected objective function has not fallen by a relative amount of RELPHIRED over NPHIRED successive iterations, then RSI_HP ceases execution.

4.5 Regularisation

RSI can read a PEST control file in which the PESTMODE control variable is set to “regularisation”. Regularisation can be used in two different ways. These are now described.

4.5.1 Regularisation for Ensemble Uniformity

This mode of regularisation usage encourages ensemble collapse as the inversion process progresses. However it encourages this to happen in a manner that yields an approximation to the minimum error variance solution to an inverse problem.

It is important to note that the extent to which RSI can achieve a minimum error variance solution to an inverse problem is limited by the number of realizations that comprise an ensemble. These realizations are generally random, and not spatially smooth. Hence, while their use often induces a good fit with a calibration dataset after a stunningly small number of model runs, estimated parameter fields may possess “bumps and lumps” which compromise the “simplicity” or “smoothness” which characterizes a true minimum error variance solution to an inverse problem. Nevertheless, a parameter field obtained in this way may constitute a useful alternative to a true minimum error variance parameter field if:

- Prior information embodying Tikhonov constraints is included in the PEST control file on which the RSI inversion process is based; and
- The ensemble which is used by RSI is comprised of a sufficiently high number of realisations for random bumps and lumps in individual parameter fields to “cancel” as random realizations are combined.

When the normal version of PEST is run in “regularisation” mode, PEST applies a weight factor to all regularisation observations (which are normally prior information equations). This weight factor is calculated internally by PEST such that the inversion process seeks a user-specified target measurement objective function which is supplied as the PHIMLIM regularisation control variable. Furthermore, if the IREGADJ regularisation control variable is set to a number greater than zero, then PEST varies the weight factor between different regularisation groups in order to support stronger application of regularisation constraints on parameters which are relatively uninformed by a calibration dataset.

RSI_HP’s implementation of regularisation is not quite the same as that of PEST. RSI_HP ignores both PHIMLIM and IREGADJ. The only variable from the “regularisation” section of the PEST control file that RSI_HP reads is REG2MEASRAT. As is described in documentation of PEST_HP, this variable sets the ratio of the regularisation objective function to the measurement objective function. During any iteration of the inversion process, RSI_HP updates the regularisation weight factor such that this ratio is respected. That is, it calculates a weight factor for which the regularisation component of the total objective function is equal to REG2MEASRAT times the measurement component of the total objective function. A value of 0.1 or 0.05 is often suitable for REG2MEASRAT.

Because RSI_HP does not use the IREGADJ control variable to balance weights between different regularisation groups, it is incumbent on a user to pay attention to relativity of weighting between these groups when building a PEST control file. This is particularly important when regularisation is supplied through prior information equations which are accompanied by covariance matrices. As is described elsewhere in PEST documentation, covariance matrices are normally based on variograms. The variogram that is associated with each parameter type should reflect anticipated spatial variability of that parameter type throughout the model domain.

If weights are supplied for some prior information equations instead of a covariance matrix, then these too should reflect expected parameter variability. Recall that a weight should denote the inverse of the standard deviation of variability of the “observed value” of a prior information equation. (The value of a prior information equation is often the value of a single parameter when prior information is used for regularisation.) Standard deviation is the square root of variance. In contrast, diagonal elements of a covariance matrix reflect parameter variances instead of parameter standard deviations; these are equal to the sill of the variogram from which the covariance matrix is derived.

As for PEST and PEST_HP, RSI_HP reports two objective functions when it is run in “regularisation” mode. One of these is the measurement objective function, while the other is the regularisation objective function. The former is calculated using all non-regularisation observation groups, while the latter is calculated using only regularisation groups. It is important to note, however, that the internally-calculated regularisation weight factor is not used to calculate the regularisation objective function that PEST and RSI_HP reports to the screen and to their run record files. This has a number of repercussions. In particular:

- The regularisation objective function reported by RSI_HP is a measure of adherence to regularisation constraints; this measure does not vary from iteration to iteration. It is directly comparable to the regularisation objective function reported by other members of the PEST suite such as PEST_HP.
- At any stage of the inversion process, the sum of the measurement objective function and the regularisation objective function does not equal the total objective function. However, the sum of the measurement objective function and the current regularisation weight factor times the current regularisation objective function is equal to the total objective function.

When RSI_HP runs in “regularisation” mode, it reports the total objective function to the screen and to its run record file, just as it does when the PESTMODE variable is set to “estimation”. RSI_HP minimizes this total objective function. However, during each iteration of its history-matching process, RSI_HP also reports the measurement objective function associated with realizations for which the minimum and maximum total objective functions were attained. It also reports the minimum, maximum and average values of the measurement objective function over all realizations, together with corresponding regularisation objective functions.

At the end of an RSI-based inversion process, you may decide that the measurement objective function is too low, and that estimated parameter fields embodied in realisations which comprise the collapsed ensemble are too “bumpy” because of overfitting. In this case you are free to select your “calibrated” parameter field from realizations achieved during previous iterations of the inversion process where measurement objective functions were higher and regularisation objective functions were lower. The sequence of solutions attained during the iterative RSI_HP inversion

process thus constitutes a primitive means of exploring a kind of “pareto front” in which the measurement objective function is traded off against the regularisation objective function.

4.5.2 Regularisation for Solution Divergence

Regularization can also encourage realizations to stay as different as possible as they are simultaneously varied in order for the model to fit the history-matching dataset. This mode of regularization usage is similar to that employed by a traditional ensemble smoother.

RSI_HP behaves in this manner provided the following conditions are met:

- A prior information equation is supplied for each parameter;
- Each of these prior information equations provides a “preferred value” for the pertinent parameter;
- The PESTMODE control variable is set to “regularisation”;
- The REGDIVERSE variable in the “rsi” section of the PEST control file to 2;
- Appropriate values are supplied for the REGDIVFRAC and MINREGWTFAC RSI control variables (these variables will be described shortly).

The first two of the above conditions are easily met if prior information is added to a PEST control file using the ADDREG1 or ADDREG2 utilities supplied with PEST.

When “diversity regularisation” is implemented in the way described above, RSI_HP provides realization-specific “observed” values for all prior information equations. These observed values are equal to the initial values of parameters comprising all realisations. Thus parameters featured in each realization are discouraged from departing from their initial values.

The REGDIVFRAC RSI control variable has the same role as that of the REG2MEASRAT regularisation control variable. That is, it sets the ratio of regularisation to measurement objective functions that RSI_HP should employ during each iteration of the inversion process. The value supplied for REGDIVFRAC in the “rsi” section of a PEST control file overrides the value supplied for REG2MEASRAT in the “regularisation” section of the same PEST control file. However while RSI_HP is seeking to minimize the total objective function, the regularisation objective function should be prevented from falling to a value which is too low, for this may precipitate ensemble collapse as all realizations converge towards an overfitted value of the measurement objective function. This can be prevented by providing an appropriate value for the MINREGWTFAC control variable. MINREGWTFAC denotes the lowest permissible value for the regularisation weight factor. That is, this is the lowest value by which prior information weights supplied in a PEST control file (or the elements of the square roots of inverted covariance matrices supplied in their stead) will be multiplied as the measurement objective function falls. The optimum value for MINREGWTFAC is context-specific. A value between 0.5 to 0.7 is often suitable. However optimality of this choice is predicated on the assumption that weights and/or covariance matrices associated with prior information equations reflect true “prior” variability of respective parameter types. As stated above, the user should try to achieve this when building the PEST control file.

When regularization is used to promote ensemble diversity in this way, RSI records total and measurement objective functions on the screen and on its run record file, just as it does when regularisation is used to promote strategic ensemble collapse. However it does not record the regularisation objective function under these circumstances, as this loses its meaning.

4.6 RSI_HP Outputs

4.6.1 Screen

During each iteration of the inversion process, RSI_HP writes information in the working command line window. This allows a user to monitor its progress. This information includes the following:

- The number of realizations that are currently being used, and whether any realizations have been abandoned;
- The average, minimum and maximum objective functions calculated over all realizations, and the realization names (parameter and observation) associated with the last of these two;
- The average, minimum and maximum “actual” objective functions over all realizations (see above text for discussion of “actual”);
- The outcomes of lambda testing conducted on realizations that were selected for the lambda-testing process.

4.6.2 Run Record File

All information that is recorded on the screen is also recorded in the RSI_HP run record file. This file is named *case.rec*, where *case* is the filename base of the PEST control file. The run record file begins with a listing of values for RSI_HP control variables provided by the user in the PEST control file. Note, however, that it does not echo other information supplied in the PEST control file.

4.6.3 Ensemble Output Files

During each iteration of the ensemble inversion process, RSI_HP records files named *case.par.N.jcb* and *case.obs.N.jcb* (where *case*, once again, is the filename base of the PEST control file and *N* is the iteration number starting at 1). The first of these files contains the values of all parameters comprising all current realizations, while the second of these files lists the model-generated counterparts to observations calculated using respective realizations. The names associated with parameter and observation realizations are those provided by the user in the respective initial ensemble files; abandoned ensembles are omitted from these JCB files.

A JCB file can be converted to a CSV file (i.e. a comma-delimited file) using the JCB2CSV utility supplied with the PEST suite. Other utilities for handling and manipulating JCB files are also provided in the PEST utility suite.

4.6.4 Phi Output Files

During each iteration of the inversion process, RSI_HP writes two CSV files which record objective functions calculated for each parameter/observation realization pair which remains in the total ensemble of realizations (i.e. each realization pair that has not been abandoned because of model run failure or inability of RSI_HP to lower the objective function). These files are named *case.N.phi.csv* and *case.N.phi.actual.csv* where *case* is the filename base of the PEST control file and *N* is the iteration number. As the name implies, the first records objective functions while the second records actual objective functions; see section 4.4.10 for the difference between these two types of objective function. Objective functions components for each observation group are also recorded in these files.

RSI_HP records another objective function CSV file. This file is named *case.phi.summary.csv*. This file records objective function statistics. Information is appended to this file at the end of each iteration

of the RSI_HP inversion process. It thereby provides a useful summary of the progress of the ensemble-based inversion process at any time during that process.

4.6.5 Relating File and Screen Output

It is important to understand how the contents of files *case.N.par.jcb*, *case.N.obs.jcb*, *case.N.phi.csv* and *case.N.phi.actual.csv* relate to RSI_HP screen outputs and to the contents of file *case.phi.summary.csv*.

During each iteration of its inversion process, RSI_HP undertakes the following tasks.

1. First, RSI_HP runs the model once for each parameter realization, except for those realizations that were selected for Marquardt lambda testing during the previous iteration as model runs have already been undertaken for these realizations.
2. If secondary realizations are allowed to alter primary realizations during this particular iteration, RSI_HP also runs the model once for each secondary realization.
3. It calculates objective functions for all realizations.
4. RSI_HP rejects realization updates or abandons realizations if necessary.
5. It records objective function summaries to the screen and to file *case.phi.summary.csv*.
6. It records the current parameter ensemble in file *case.N.par.jcb* and corresponding model outputs in file *case.N.obs.jcb*.
7. It records realization-specific objective functions in files *case.N.phi.csv* and *case.N.phi.actual.csv*.
8. RSI_HP then calculates NUMLAMTEST Marquardt lambdas.
9. It then undertakes NUMREALLAM $\times$ NUMLAMTEST model runs to test these lambdas.
10. It calculates objective functions pertaining to these lambda-testing model runs so that it can decide on the most effective lambda value to use in upgrading remaining realizations; RSI_HP reports the best objective function for each tested realization to the screen and to its run record file.
11. Finally, using the best lambda, RSI_HP calculates upgrades for all other realizations.

For those iterations in which supplementary realizations are read, the upgrading of existing realizations using these supplementary realizations replaces steps 8 to 11 of the above sequence.

Parameter realizations which exist at the start of iteration N are thus those which were updated at the end of the previous iteration. The objective functions corresponding to all of these (except for those which were involved in lambda testing during the previous iteration) have not been calculated at the start of a realization. Commissioning of model runs to calculate these objective functions is the first task undertaken during that iteration. However, for iteration number 1, the realizations for which model runs are undertaken by RSI_HP are those which were read from the user-supplied parameter ensemble file; obviously, none of these realizations have been used for lambda testing. These same initial realizations are recorded in file *case.par.1.jcb*; model outputs corresponding to these initial realizations are recorded in file *case.obs.1.jcb*. The objective functions corresponding to these realizations are associated with iteration 1 in RSI_HP screen output and in file *case.phi_summary.csv*.

In summary, parameter realizations and objective functions which are associated with iteration N constitute the outcomes of $N-1$ realization updates.

4.7 RSI Control Variables

4.7.1 PEST Control File

As has already been discussed, RSI_HP reads a PEST control file. Information that it gleans from this file includes the following:

- Parameter names, initial values, transformation status, bounds, scales and offsets;
- Observation names, weights, groups and (optionally) observation standard deviations;
- The command to run the model;
- The names of template and instruction files, and of corresponding model input and output files;
- The values of some PEST control variables.

When reading this file, RSI_HP will cease execution with an appropriate error message if it finds that the value supplied for a PEST control variable is in error or does not make sense.

4.7.2 Traditional PEST Control Variables

RSI_HP obtains values for most of its control variables from the “rsi” section of the PEST control file. However, it uses some of the other variables which it reads from a PEST control file as well. These variables are listed in table 4.1.

Table 4.1 Traditional PEST control variables employed by RSI_HP. Where pertinent, recommended values for use with RSI_HP are shown in *italics*.

Variable	Section	Role in RSI_HP
NPAR	control data	number of parameters
NOBS	control data	number of observations
NPRIOR	control data	number of prior information equations
NOBSGP	control data	number of observation groups
NTPLFE	control data	number of template files
NINSFLE	control data	number of instruction files
PRECIS	control data	format for writing parameters to model input files
DPOINT	control data	omit decimal point when writing parameter values if possible
NOPTMAX	control data	maximum number of iterations
EIGHTHRESH	singular value decomposition	truncation point set by singular value ratio (<i>5E-7 is suggested</i>)
APAR	parameter data	parameter name
PARTRANS	parameter data	parameter transformation status
PARCHGLIM	parameter data	type of change limit applied to parameter (<i>Note that a change limit of “absolute” is not allowed by RSI_HP.</i>)
PARLBND	parameter data	lower parameter bound
PARUBND	parameter data	upper parameter bound
SCALE	parameter data	scale applied to parameter
OFFSET	parameter data	offset applied to parameter
OBSNME	observation group	name of observation group
COVFLE	observation group	name of a file that holds a covariance matrix
OBSNME	observation data	name of observation
OBSVAL	observation data	value of observation
WEIGHT	observation data	weight associated with observation
STANDARD_DEVIATION	observation data	standard deviation of measurement noise
COMLINE	model command line	command to run model (<i>only one command is allowed</i>)
TEMPFLE	model input/output	name of template file
INFLE	model input/output	name of model input file
INSFLE	model input/output	name of instruction file
OUTFLE	model input/output	name of model output file
PILBL	prior information	name of a prior information equation
PIFAC	prior information	parameter factor in prior information equation

PARNME	prior information	parameter name in prior information equation
PIVAL	prior information	value of prior information equation
WEIGHT	prior information	weight associated with prior information equation
OBGNAME	prior information	observation group name to which prior information equation belongs

4.7.3 RSI Section

The PEST control file read by RSI_HP must contain a section named “rsi”. In accordance with normal PEST protocol, the header which leads this section must include the name of the section preceded by an asterisk followed by a space.

As the name implies, the “rsi” section of a PEST control file contains RSI control variables. These are identified by their positions on each line of this section. The “rsi” section should be placed somewhere between the “control data” and “parameter groups” sections of a PEST control file; its ordering with respect to other sections which can be placed between the “control data” and “parameter groups” sections of a PEST control file is arbitrary.

Specifications for the “rsi” section of a PEST control file (including the positions of all control variables) are provided in figure 4.3.

```
* rsi
RANDOMSEED
NREAL INCLUDE_BASE LOSE_IF_NO_IMP LOSE_PHI_QUANTILE READ_OBS_ENSEMBLE [NMDA]
RSI_RELPARMAX RSI_FACPARMAX SUPPARMAX IBOUNDPT REJECT_UPDATE REJECT_THRESHOLD
NUMREALLAM NUMLAMTEST REALLAMSELECT
REGDIVERSE REGDIVFRAC
PHIREDTYPE RELPHIRED NPHIRED
NREALSEC [ASYMSEC NUMITERSEC SECITER(1) SECITER(2) SECITER(3)...]
USESUPINT [MAXSUPITN SUPREPLACE SUPTHRESHVAL SUPFRAC TEMP_REJECT_THRESHOLD]
PARENSEFILE
OBSENFILE
PARENSECFILE
PARENSUPFILE
```

Figure 4.3 Variables appearing in the “rsi” section of a PEST control file.

An example of the “rsi” section of a PEST control file is provided in figure 4.4.

```
* rsi
1111 # RANDOMSEED
80 0 2 0.9 -1 4 # NREAL INCLUDE_BASE LOSE_IF_NO_IMP LOSE_PHI_QUANTILE READ_OBS_ENSEMBLE NMDA
120.0 120.0 100.0 1 1 2.0 # RSI_RELPARMAX RSI_FACPARMAX SUPPARMAX IBOUNDPT REJECT_UPDATE REJECT_THRESHOLD
5 10 1 # NUMREALLAM NUMLAMTEST REALLAMSELECT
0 0.10 0.5 # REGDIVERSE REGDIVFRAC MINREGWTFAC
1 0.1 2 # PHIREDTYPE RELPHIRED NPHIRED
10 0 8 3 4 5 6 7 8 9 10 # NREALSEC ASYMSEC NUMITERSEC SECITER(1) SECITER(2) etc
0 6 0 1E30 0.5 20.0 # USESUPINT MAXSUPITN SUPREPLACE SUPTHRESHVAL SUPFRAC TEMP_REJECT_THRESHOLD
init_par_ensemble.jcb # PARENSEFILE
init_obs_ensemble.jcb # OBSENFILE
sec_par_ensemble.jcb # PARENSECFILE
init_par_ensemble.big.jcb # PARENSUPFILE
```

Figure 4.4 An example of the “rsi” section of a PEST control file.

Reference was made to many of the variables appearing in the “rsi” section of a PEST control file in section 4.4 of this document. They are now discussed in greater detail.

RANDOMSEED

Provide any positive integer. RSI_HP uses random number generation for selection of realizations for lambda testing if the REALLAMSELECT control variable is set to 1, and for generation of realisations of measurement noise if READ_OBS_ENSEMBLE is set to -1.

NREAL and INCLUDE_BASE

NREAL specifies the number of realizations that RSI_HP employs. This is the dimensionality of realization space. (Note that NREAL does not include secondary realizations; it only specifies the number of primary realizations.) The number of parameter realizations contained in the PARENSFILE JCB file, and the number of observation realizations contained in the OBSSENSFILE JCB file must be greater than or equal to NREAL, unless INCLUDE_BASE is set to 1, in which case the number of parameter/observation realizations contained in these files must be greater than or equal to NREAL-1. If INCLUDE_BASE is set to 1, then RSI_HP assigns parameter values featured in the “parameter data” section of the PEST control file to a parameter realization named “base”, and observation values featured in the “observation data” section of the PEST control file to an observation realization named “base”.

LOSE_IF_NO_IMP and LOSE_PHI_QUANTILE

If the objective function associated with a realization does not improve over LOSE_IF_NO_IMP successive iterations, then that realization is abandoned. However it is only abandoned if the objective function associated with that realization is in the upper LOSE_PHI_QUANTILE quantile of objective functions. For example, if LOSE_IF_NO_IMP is set to 4 and LOSE_PHI_QUANTILE is set to 0.9, then if the objective function associated with a particular realization shows no improvement over 4 successive iterations, and if, after those 4 iterations, the objective function associated with that realization is in the top 10 percent of objective functions, the realization is abandoned.

Set LOSE_IF_NO_IMP to 0 to disable realization abandonment. If setting it to a non-zero value, keep in mind the fact that objective functions are not likely to improve in those iterations in which supplementary realizations are read (which will only occur if the USESUPINT control variable is set to a positive value.)

Abandonment of non-performing realizations can improve the integrity of the super parameter Jacobian matrix which RSI_HP uses to improve each realization. However this improvement can come at a cost. Diversity of the ensemble may be lost, especially if many realizations are abandoned. At the same time, with the dimensionality of realization space reduced, the inversion process may not achieve as good a fit with the calibration dataset as it otherwise could.

READ_OBS_ENSEMBLE

READ_OBS_ENSEMBLE is an integer. It must be set to 1, -1 or 0. If set to 1, RSI_HP reads realizations of measurement-noise-enhanced observations from an observation ensemble file. However if it is set to 0, then RSI_HP does not read an ensemble of observation values from an observation ensemble file. Instead, each observation “realization” is the same as the set of observations that reside in the PEST control file. No noise is added to these observations before attempting to fit them.

If READ_OBS_ENSEMBLE is set to -1, then RSI_HP generates observation realisations itself. If the PEST control file is accompanied by an external observation data file, then RSI_HP reads standard deviations of observation noise from the final column of this file. If an external observation data file is not provided, then RSI_HP assumes that the weight assigned to each observation is the inverse of the standard deviation of measurement noise associated with that observation.

NMDA

If NMDA is set to a value greater than 1, then RSI_HP undertakes multiple data assimilation. This is done for the first *NMDA* iterations of the ensemble inversion process that RSI_HP undertakes. Note that if NMDA is set to a value greater than 1, then READ_OBS_ENSEMBLE must be set to -1, so that RSI_HP generates its own observation ensemble.

RSI_RELPARMAX and RSI_FACPARMAX

These are equivalent to the PEST RELPARMAX and FACPARMAX control variables. However they are duplicated in the “rsi” section of the PEST control file so that a user does not forget to supply values for them that are best for use with RSI_HP. At the time of writing, values of at least 100.0 seem suitable for each of these variables.

SUPPARMAX

SUPPARMAX sets the maximum allowable change that any parameter is allowed to undergo when a super parameter (i.e. a realization) is adjusted; parameter log-transformation is taken into account when computing this change. At the time of writing, a value of 100.0 is suggested for this variable.

IBOUNDOPT

Through IBOUNDOPT (an integer variable), an RSI_HP user informs RSI_HP how it should handle parameter bounds. Note that bounds constrain parent parameters only. Tied parameters maintain their original ratios with parent parameters, even if adherence to these ratios causes them to transgress their bounds.

An IBOUNDOPT setting of 0 instructs RSI_HP to ignore parameter bounds altogether as parameter realizations are updated. In contrast, when IBOUNDOPT is set to 1, parameters that encounter their bounds are clipped at those bounds. An IBOUNDOPT setting of 2 prevents the clipping of parameters by instructing PEST_HP to shorten realization update vectors. This preserves parameter relatively within each realization, but may reduce the ability of the realization update process to lower objective functions. If IBOUNDOPT is set to 3, RSI_HP adjusts realizations comprised of parameters which are transformed according to equation 4.6. Under some circumstances, particularly where parameters show spatial correlation, this introduces distortions to parameter relativity; these distortions may make it difficult for RSI_HP to reduce objective functions.

REJECT_UPDATE and REJECT_THRESHOLD

REJECT_UPDATE is an integer variable. Set it to 0 to prevent the rejection of realization updates. Alternatively, if REJECT_UPDATE is set to 1, then if any realization suffers an objective function increase by greater than a factor of REJECT_THRESHOLD of its existing value, the updated realization is rejected and the existing realization is retained. At the time of writing, settings of 1 for REJECT_UPDATE and 1.5 to 2.0 for REJECT_THRESHOLD are recommended.

NUMREALLAM, NUMLAMTEST and REALLAMSELECT

All three of these control variables affect the way in which a search is made for the optimal Marquardt lambda to use during each iteration of the inversion process. NUMLAMTEST sets the number of Marquardt lambdas that are tested, while NUMREALLAM sets the number of realizations that are used for testing each of these lambdas. Thus the number of model runs that are committed to lambda testing is NUMLAMTEST × NUMREALLAM; for parallelization efficiency this product should be a multiple of the number of agents that are being used to carry out model runs.

REALLAMSELECT (an integer variable) informs RSI_HP how it should select realizations for lambda testing. A setting of 1 tells RSI_HP to select them randomly. A setting of 2 informs RSI_HP that it should select the NUMLAMTEST realizations whose current objective functions are highest.

REGDIVERSE, REGDIVFRAC and MINREGWTFAC

REGDIVERSE and REGDIVFRAC control the operation of diversity regularization. REGDIVERSE is an integer variable that should be set to 1 or 2 to enable this type of regularization; set it to 0 to disable diversity regularization. When REGDIVERSE is set to 1, RSI_HP calculates a “diversity objective function” that measures departure of the separation between realizations from the original average realization separation. Weights are ascribed to inter-realization separation residuals such that the “diversity objective function” is equal to a fraction REGDIVFRAC of the current “measurement objective function” (i.e. the usual objective function) associated with any realization. As has already been stated, this type of diversity regularization can reduce the speed with which RSI_HP lowers model-to-measurement misfit. Hence, at the time of writing its use is not generally recommended. However, if it is used, a value of 0.05 to 0.1 is suggested for REGDIVFRAC.

If PEST is run in “regularisation” mode, then REGDIVERSE can be set to 2 to implement a different type of diversity regularisation. In this case, prior information is used to constrain parameters to depart minimally from their realisation-specific initial values; one or more covariance matrices can accompany these equations. REGDIVFRAC should then be used in place of the REG2MEASRAT regularisation control variable to set the ratio of regularisation to measurement objective functions.

If REGDIVERSE is set to 2, then a value is required for MINREGWTFAC. This is the minimum weight factor applied to regularisation. If observation weights are “correct” (i.e. if they are the inverse of the standard deviations of measurement noise) and the weights and/or covariance matrices ascribed to regularisation observations and prior information equations are correct, then MINREGWTFAC should probably be set to about 0.5. However some trial and error may be required to getting this right.

PHIREDTYPE, RELPHIRED and NPHIRED

These are termination criteria applied to either the average objective function across all realizations, the minimum objective function across all realizations, or the maximum objective function across all realizations. The choice between these alternatives is set by the integer PHIREDTYPE variable; this must be set to 1 for average, 2 for minimum, or 3 for maximum. If NPHIRED iterations pass without the selected objective function falling by more than a fraction RELPHIRED of its value at the start of each iteration, then RSI_HP terminates execution.

Suggested values for PHIREDTYPE, RELPHIRED and NPHIRED are 1, 0.05 and 3. Note, however, that if supplementary realizations are read, it may be wise to set NPHIRED greater than this as the introduction of supplementary realizations may temporarily cause objective functions to rise.

As has already been discussed, the NOPTMAX variable from the “control data” section of a PEST control file can also be used to control termination of RSI_HP execution. If NOPTMAX is set to N , RSI_HP will terminate execution during iteration $N+1$ just after it has run the model the number of times required for it to associate objective functions with all realization updates computed during the previous iteration.

NREALSEC, ASYMSEC, NUMITERSEC, SECITER(1), SECITER(2) etc

NREALSEC is an integer variable. If set to 0, the use of secondary realizations is disabled. All other entries on this same line are then ignored. However, if it is set to a positive number, NREALSEC indicates the number of secondary parameter realizations that are employed in the current RSI_HP inversion process. Generally this should be less than the number of primary parameter realizations (often significantly less). Secondary realizations are read from the file whose name is ascribed to the PARENSECFIL control variable.

ASYMSEC, an integer variable, should be set to -1, 0 or 1. RSI_HP calculates the factor by which each secondary parameter realization should be multiplied before being added to a particular primary parameter realization. If ASYMSEC is set to 1 then only positive factors are accepted; if it is set to -1 then only negative factors are accepted. An ASYMSEC setting of zero allows both positive and negative factors to be accepted.

There is no need for the use of secondary parameter realizations to be activated during every iteration of the RSI_HP ensemble adjustment process; they only form part of the RSI_HP inversion process during use-specified iterations. The iterations in which they are featured are specified by variables NUMITERSEC, SECITER(1), SECITER(2), etc. NUMITERSEC specifies how many RSI_HP iterations will feature secondary realizations. The numbers that follow NUMITERSEC must nominate those realizations. Obviously, the number of entries that follow NUMITERSEC must be equal to NUMITERSEC.

USESUPINT, MAXSUPITN, SUPREPLACE, SUPTHRESHVAL, SUPFRAC and TEMP_REJECT_THRESHOLD

USESUPINT (an integer variable) must be set to 0, or to a number greater than 1. If set to a number greater than 1, then RSI_HP reads supplementary parameter realizations on iterations that are determined by its value, as well as by the value assigned to the MAXSUPITN integer control variable. Suppose that USESUPINT is set to 3 and that MAXSUPITN is set to 9. Then RSI_HP will read supplementary realizations on the 3rd and 6th iterations of the inversion process, but not on any iterations after that, for MAXSUPITN sets the iteration number at which RSI_HP stops reading supplementary realizations.

As has already been explained, supplementary parameter realizations do not add to the realization pool that comprises the ensemble on which RSI_HP's attention is focussed. The purpose of supplementary realizations is to modify existing realizations. Suppose that the number of existing realizations employed by RSI_HP is NREAL. Then, on pertinent iterations, RSI_HP reads the next NREAL supplementary realizations from a user-supplied supplementary parameter realization file. (The name of this file is ascribed to the PARENSUPFILE control variable.) If, during any of these realization-reading episodes, RSI_HP runs out of realizations to read from this file, it ceases execution with an appropriate error message.

Each supplementary realization is associated with an existing realization on a sequential basis. The role of each supplementary realization is to modify its partnered existing realization; however modification takes place only for supplementary parameter values that are lower in absolute value than SUPTHRESHVAL, a real variable. Thus the supplementary realisations can be restricted to a user-designated portion of the overall model domain.

As they are read by RSI_HP, all parameter values comprising supplementary realizations are multiplied by SUPFRAC. (This is normally set to 1.0.) However parameter values which exceed SUPTHRESHVAL are not multiplied, and thus retain their original above-SUPTHRESHVAL values; they are then ignored by RSI_HP. Alternatively, if the value of a supplementary parameter is less than SUPTHRESHVAL, then this value (multiplied by SUPFRAC) either replaces, multiplies or adds to the complementary parameter value in its partnered existing realization. Its action is governed by the value of the integer SUPREPLACE control variable. If SUPREPLACE is set to 1, then the supplementary parameter value replaces the existing parameter value. Alternatively, if SUPREPLACE is set to 0, then the supplementary parameter value is added to the existing parameter value in the partnered realization if the PARTRANS status of that parameter is “none”; otherwise it multiplies the existing parameter value. Note that parameters that are declared as “fixed” in the PEST control file are unaffected by this operation; meanwhile tied parameters are adjusted to maintain their existing ratios with parent parameters after alterations to the latter parameters are incurred through use of supplementary parameters.

During those iterations in which supplementary parameter realizations are read, existing parameter realizations undergo no alterations other than those pertaining to the action of supplementary realizations. In particular, they are not adjusted using equation 4.5 in order to lower objective functions. Because existing realizations have been modified in a somewhat random way through the action of supplementary realizations, their objective functions will probably increase. If realization upgrade rejection is enabled, many of these alterations may therefore be rejected; therefore many adjusted realizations will be replaced by those which prevailed prior to the reading of supplementary realizations. In order to forestall this occurrence, RSI_HP allows the value of the REJECT_THRESHOLD variable to be temporarily raised during this iteration. This temporary value is supplied as the value of the TEMP_REJECT_THRESHOLD control variable.

If USESUPINT is set to 0, then there is no need to supply the values of any other variables on this line of the PEST control file.

PARENSFILE and OBSENSFILE

At the commencement of its execution, RSI_HP reads NREAL or NREAL-1 parameter realizations from the file whose name is ascribed to the PARENSFILE character variable. Then, if READ_OBS_ENSEMBLE is set to 1, it reads NREAL or NREAL-1 observation realizations from a file whose name is ascribed to the OBSENSFILE control variable. Both of these files must be of the JCB type.

Note that if READ_OBS_ENSEMBLE is set to 0 or -1, then RSI_HP does not read an observation ensemble from file OBSENSFILE. Nevertheless, a name must be supplied for file OBSENSFILE in the PEST control file, despite the fact that this name is ignored by RSI_HP.

PARENSECFILE

If the use of secondary realizations is enabled, RSI_HP reads NREALSEC secondary parameter realizations from the file whose name is ascribed to the PARENSECFILE control variable at the commencement of its execution. This file must be of the JCB type. If no secondary realizations are read, then a name must nevertheless be ascribed to this variable; under these circumstances it is ignored by RSI_HP.

PARENSUPFILE

This control variable must be assigned the name of the file from which supplementary parameter realizations are read. If no supplementary realizations are read, then a name must nevertheless be ascribed to this variable; under these circumstances it is ignored by RSI_HP.

4.8 Supporting Utilities

A number of utility programs that are supplied with the PEST suite can be used to assist in preparation for an RSI_HP run, and in postprocessing outputs from that run. The number of these utilities is expected to increase over time. The RANDOBS, RANDPAR3, RANDPAR4 and JCB2CSV utilities have already been mentioned. Other PEST utilities can be used for reading, writing and extracting information from JCB files. See part II of the PEST manual for documentation of these utilities.

These same utilities can also be used to prepare input files for the PESTPP-IES iterative ensemble smoother program, and to read some of its output files. PESTPP-IES, and other members of the PEST++ suite, can be downloaded from the following site:

<https://github.com/usgs/pestpp>

5. Ensemble Space Inversion

5.1 Introduction

Ensemble space inversion (ENSI) resembles realisation space inversion (RSI – see the previous chapter) because it combines random realisations of parameters drawn from a prior parameter probability distribution to derive parameter sets that allow model outputs to match field measurements. A further commonality with RSI is that a high level of model-to-measurement fit can often be achieved with relatively few model runs – often far fewer than the number of parameters with which a model is populated. However, there are some important differences between ENSI and RSI.

One important difference is that, unlike RSI, ENSI does not seek a suite of parameter fields that promote a good fit between model outputs and a calibration dataset. Instead, it seeks a single parameter field which achieves a high level of model-to-measurement fit. In doing so, it applies regularisation constraints in order to ensure that this parameter field is the closest approximation to the minimum error variance solution to the inverse problem that can be achieved within the limited dimensional parameter space in which it is obliged to operate.

5.2 Concepts

5.2.1 Ensemble Smoothers

PEST users have ready access to ensemble smoother technology through the powerful PESTPP-IES package that is supplied with the PEST++ suite.

ENSI bears some similarities to PESTPP-IES in that the history-matching process that it implements commences with parameter realisations drawn from the prior parameter probability distribution. PESTPP-IES alters all of these realisations until they collectively sample an approximation to the posterior parameter probability distribution. Unless localisation is applied, the altered parameter fields that it calculates are linear combinations of the original parameter fields.

This process of linear parameter set combination can be expressed symbolically by considering a single sample of the final PESTPP-IES parameter ensemble. If we denote this i 'th realisation of the final parameter ensemble as $\mathbf{j}_i$. Then:

$$\mathbf{j}_i = \mathbf{K}\mathbf{f}_i \quad (5.1)$$

In equation 5.1, each column of the $\mathbf{K}$ matrix is a parameter realisation sampled from the prior parameter probability distribution. The vector $\mathbf{f}_i$ specifies how these columns are combined to form $\mathbf{j}_i$. That is, it contains factors by which each of the columns of $\mathbf{K}$ is multiplied to form $\mathbf{j}_i$. Other $\mathbf{f}$'s (for example $\mathbf{f}_j$, $\mathbf{f}_k$ etc.) hold factors that are used to compute other $\mathbf{j}$'s (for example, $\mathbf{j}_j$, $\mathbf{j}_k$ etc.).

Equation 5.1 (and equations that follow) omit mean parameter values. For the moment, these are assumed to be zero, so that parameters are defined as departures from their prior means. This matter is further discussed below.

In common with other ensemble smoothers, PESTPP-IES does not actually use equation 5.1 as a basis for history-matching. Instead, it calculates an approximate, rank-deficient Jacobian matrix that links

observation-pertinent model outputs to individual model parameters. This approximate Jacobian matrix is calculated from empirically-assessed covariances between model outputs and parameters evaluated over N model runs, where N is the number of realisations comprising an ensemble. Unless localisation is employed, the rank of this Jacobian matrix is the same as the number of realisations that is used to compute these covariances, and hence the number of columns of $\mathbf{K}$.

Easy and effective implementation of localisation is a strength of most implementations of the iterative ensemble smoother. Localisation increases the rank of the Jacobian matrix on which parameter adjustment is based. Hence $\mathbf{j}$ vectors that emerge from the localisation-enhanced inversion process are no longer linear combinations of parameter realisations comprising the columns of $\mathbf{K}$. This allows better fits to be attained between model outputs and field measurements. At the same time, emergent parameter fields are often more “realistic”, as a limited number of parameter realisations do not have to be “bent into shape” in order for model outputs to match measured values of system states and fluxes.

While the advantages of localisation are obvious, localisation must be used with caution. Its use is often based on the premise that a parameter cannot effect a model output that is geographically distant from it, or that its effect on model outputs wanes with distance in a manner that can be emulated by crude alterations to an empirical covariance matrix. Unfortunately, there are many situations in groundwater modelling where geographic separation does not imply diminished sensitivity. For example, parameters that govern hydraulic properties of a model boundary may effect all model outputs that are upstream of the boundary, regardless of their distance from it.

5.2.2 ENSI

While the ENSI algorithm does not accommodate localisation, it can make efficient and flexible use of the subset of parameter space in which it must operate.

Suppose that a PEST control file cites *NPAR* parameters. Suppose further that these are subdivided into *NPARGP* parameter groups. Suppose also that the following requirements are met.

1. The inverse problem (including parameters and observations) is defined in a PEST control file that instructs PEST to run in “estimation” mode.
2. Prior parameter uncertainties are ascribed to model parameters. These can be comprised of one or a number of covariance matrices that pertain to parameter groups, and one or a number of standard deviations attributed to individual model parameters. All of these are listed in a parameter uncertainty file.
3. Initial parameter values cited in the PEST control file are preferred parameter values from a prior probability distribution standpoint.
4. No prior information is included in the PEST control file.

All of the above conditions are typical of those which precede deployment of the PESTPP-IES ensemble smoother. Hence if a modeller has prepared for a PESTPP-IES run, then he/she has also prepared for an ENSI inversion run.

As stated above, in the absence of localisation, IES seeks to combine an ensemble of parameter realisations in ways that solve an inverse problem. Where parameters are of many types (for example, hydraulic conductivity, porosity, and possibly hyperparameters that specify the nature of

the prior parameter probability distribution), then each parameter realisation includes values of all of these parameter types. That is, specific values of all of these parameter types are “locked” into each parameter realisation. This can have some unwanted consequences. Suppose, for example, that some parameter types effect some model output types more than others. For example, porosity effects speed of contaminant movement while hydraulic conductivity effects both contaminant movement and groundwater heads. Then selection of a certain linear combination of parameter realisations which promotes a good fit with one model output type, may hinder the ability of the model to fit another model output type. Compromises must then be made, possibly resulting in a diminution of overall model-to-measurement fit, the production of unrealistic parameter fields, or both. The need for compromise is an outcome of the limited dimensionality of the parameter space in which solution of the inverse problem is sought.

ENSI attempts to ameliorate this problem by allowing a user to decouple realisations pertaining to one type of parameter from those that pertain to another type of parameter. Hence the factors through which realisations are combined can be different for different parameter types. Suppose that there are three types of parameters. Then ENSI inversion allows modification of equation 5.1 to:

$$\mathbf{j} = \begin{bmatrix} \mathbf{K}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{K}_3 \end{bmatrix} \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_3 \end{bmatrix} \quad (5.2)$$

At the same time, in ways that will be discussed shortly, ENSI seeks only a single solution to the inverse problem. Hence realisation-pertinent subscripts for $\mathbf{j}$ and $\mathbf{f}$ are omitted from equation 5.2. Subscripts that appear in the above equation pertain to different parameter types instead.

In equation 5.2, the number of columns in each of the $\mathbf{K}_1$, $\mathbf{K}_2$ and $\mathbf{K}_3$ submatrices can be different. That is, the number of realisations that is associated with each parameter type can be different. This enables certain inversion efficiencies. For example, parameters which affect many observations that are spread over large areas (for example hydraulic conductivity in a certain model layer) may be embodied in many different realisations. In contrast, those which effect only a few observations in specific areas can be embodied in fewer realisations.

ENSI uses PEST (or, better still, PEST_HP) to directly estimate the elements of the $\mathbf{f}$ vectors of equation 5.2. Generally, the number of parameter realisations that is used to define the subset of parameter space in which the inversion process operates is much smaller than the number of actual model parameters. That is, the number of columns of the overall $\mathbf{K}$ matrix on the right side of equation 5.2 is generally much smaller than the number of matrix columns. Hence the number of elements of the collective $\mathbf{f}$ vector is also much smaller than the number of actual model parameters. The number of model runs per iteration required by the inversion process is roughly equal to the number of elements of $\mathbf{f}$ (or possibly twice this if using central differences for derivatives calculation) as these are the parameters that PEST “sees” when undertaking ENSI inversion.

In addition to allowing decoupling of different parameter types in formulating and manipulating parameter realisations, ENSI allows individual estimation of some native model parameters. These parameters are therefore omitted from realisations while still being featured in the inversion process. Candidates for individual estimation in this manner are generally parameters which have a

pervasive effect on model outputs. For example, a modeller may estimate a layer-wide mean value of hydraulic conductivity (or a small number of parameters which govern its spatial trend), and employ pilot points to represent spatial variability of hydraulic conductivity about this mean or trend. The former parameters are good candidates for individual estimation, while the latter are good candidates for realisation factor estimation. Similarly, a modeller may wish to estimate “iid” parameters which govern hydraulic property spatial details, together with a handful of individual parameters which determine the (possibly spatially-variable) variogram which governs hydraulic property spatial correlation. (See documentation of the FIELDGEN2D_SVA* and FIELDGEN3D_SVA* utilities from the Groundwater Utility suite). In this case, the former parameters are candidates for realisation-based estimation, while the latter parameters are candidates for individual estimation.

5.2.3 Regularised Inversion

After formulating the inverse problem as that of estimating factors comprising elements of the vector $\mathbf{f}$ together with a handful of individual parameters, the inverse problem can then be solved using PEST or PEST_HP. (The latter option is often better, especially for large, nonlinear models, particularly if Broyden parameter upgrading is employed and the UPTSTLIM control variable is set to about 20.) A single “minimum error variance” solution can be sought to this problem using preferred-value Tikhonov regularisation. In order to achieve this, PEST must be run in “regularisation” mode. Meanwhile preferred values for non-realisation parameters are set to their initial values, while preferred values for realisation factor parameters are set to zero (implying minimum departure of native model parameters from their prior means).

User-supplied prior parameter uncertainties are used to generate parameter realisations (i.e. columns of the $\mathbf{K}_i$ matrices of equation 5.2) which form the basis for realisation factor estimation. Prior parameter uncertainties are also used to formulate weights for Tikhonov regularisation constraints imposed on nonrealisation parameters.

Weights applied to regularisation constraints on realisation factor parameters are now examined.

As stated above, realisation factor parameters have preferred values of zero. This condition is embodied in a set of prior information equations, each pertaining to a single parameter. The weight that is ascribed to each equation is equal to $\sqrt{N-1}$ where N is the number of realisations associated with the parameter type, that is, the number of elements of a particular $\mathbf{f}_i$ that is featured in equation 5.2.

To demonstrate this, let $\mathbf{j}$ be a vector that hosts some or all native model parameters. Suppose that this parameter subset is characterised by a prior covariance matrix $\mathbf{C}(\mathbf{k})$. Suppose also, that in the ENSI inversion process, $\mathbf{j}$ is calculated from realisation factors as:

$$\mathbf{j} = \mathbf{K}\mathbf{f} \quad (5.3)$$

where the columns of $\mathbf{K}$ comprise realisations of parameters generated using the $\mathbf{C}(\mathbf{k})$ prior covariance matrix. From equation 5.3 the covariance matrix of $\mathbf{j}$ is:

$$\mathbf{C}(\mathbf{j}) = \mathbf{K}^t \mathbf{C}(\mathbf{f}) \mathbf{K} \quad (5.4)$$

where $C(\mathbf{f})$ is the prior covariance matrix of realisation factor parameters $\mathbf{f}$. Meanwhile, an empirical approximation to $C(\mathbf{k})$ can be calculated from samples of $\mathbf{k}$ comprising the columns of $\mathbf{K}$ using the equation:

$$C(\mathbf{k}) \approx \frac{\mathbf{K}^T \mathbf{K}}{N-1} \quad (5.5)$$

Combining equations 5.4 and 5.5 it is obvious that:

$$C(\mathbf{f}) = \frac{\mathbf{I}}{N-1} \quad (5.6)$$

where $\mathbf{I}$ is the identity matrix. It follows that regularisation constraints on estimated realisation factors can be embodied in prior information equations that assign these factors preferred values of zero and that have weights of $\sqrt{N-1}$, for this is the inverse of the standard deviation of the prior uncertainty of each such factor.

5.2.4 Two References

This short section on theory closes with two references to previous work with which the ENSI methodology that is documented herein has much in common, and from which ideas were gleaned prior to development of ENSI. The same authors may have written other papers on the subject. If you are interested, refer to the papers below and papers cited therein.

Chada, N.K., Iglesias, M.A., Roininen, L. and Stuart, A.M., 2018. Parameterizations for ensemble Kalman Inversion. *Inverse problems*, 34, 055009 (31pp).

Iglesias, M.A., Law, K.J.H. and Stuart, A.M., 2013. Ensemble methods for inverse problems. *Inverse problems*, 39, 045001 (20pp).

5.3 Practicalities

5.3.1 General

As stated above, estimation of realisation factors is undertaken using PEST or PEST_HP. A PEST input dataset for realisation factor estimation is constructed from a traditional PEST input dataset for native model parameters using a program named ENSIPREP. The role of this program resembles that of SVDAPREP which prepares a PEST input dataset for SVD-assisted parameter estimation. Parallels between ENSI and SVD-assist run deep. For SVD-assist, “super parameters” are actually factors which are applied to orthogonal unit vectors which collectively span the parameter solution space. For ENSI inversion, “super parameters” are factors applied to nonorthogonal random vectors which (it is hoped) are sufficient in number for at least one of them to be projectable onto any vector that lies in the parameter solution space. In both cases, the solution to the inverse problem is computed by multiplying respective vectors by vector-specific factors and summing. However, as already stated, when undertaking ENSI inversion, native model parameters can be estimated at the same time as “super parameters”.

A program named ENSIMOD back-calculates the values of native model parameters from parameter realisations and accompanying realisation factors. This program must be run as part of the model that is calibrated by PEST. (The equivalent utility program for SVD-assisted inversion is named PARCALC.)

Before discussing the ENSIPREP utility program, the files that this program reads are briefly described.

5.3.2 PEST Control File

A PEST control file defines an inversion problem. The PEST control file whose name is supplied to ENSIPREP must specify that PEST runs in “estimation” mode.

Other conditions must be met by a PEST control file on which ENSI inversion is based. In particular, this PEST control file must specify that only a single command be used to run the model. (Recall that PEST permits different commands to be used to run the model when calculating sensitivities with respect to different parameters.) Also, the command to run the model must end in “.bat”. This conveys to ENSIPREP that “the model” is actually a batch or script file. ENSIPREP modifies this file, adding the command to run ENSIMOD to its beginning, prior to other commands that run the actual simulator and its pre/postprocessors.

5.3.3 Parameter Uncertainty File

Specifications of a parameter uncertainty file are provided in Part 2 of the PEST manual. A parameter uncertainty file is comprised of a number of blocks. A STANDARD_DEVIATION block provides standard deviations for individual parameters. A COVARIANCE_MATRIX block cites a file containing a covariance matrix that defines uncertainties of a subset of model parameters. This block may also contain information that links elements of the covariance matrix to parameters that feature in the PEST control file. Optionally, it may include a factor by which the covariance matrix must be multiplied.

ENSIPREP uses parameter uncertainties in the same way that PESTPP-IES uses parameter uncertainties. It uses them to generate random realisations of parameters. These realisations are centred on parameter values that are contained in the PEST control file. The latter are therefore assumed to represent prior parameter means.

5.3.4 ENSI Specification File

Figure 5.1 exemplifies an ENSI specification file.

```
* realisation group 1
pargroup=k1
pargroup=k2
pargroup=k3
numreal=20
parameter=ghbcond1
parameter=ghbcond2

* realisation group 2
pargroup=por1
pargroup=por2
pargroup=por3
numreal=10

* nonrealisation parameters
p_global
k_global
```

Figure 5.1 An ENSI specification file.

An ENSI specification file is subdivided into sections. Each section begins with a header. The header is comprised of a "*" character followed by a space followed by either "realisation group N " (where N is an integer) or "nonrealisation parameters". Blank lines and comments can appear anywhere within the file. Comments begin with "#". These can be placed anywhere on any line of the file.

Realisation groups must be sequential. That is, the header for realisation group 1 must precede the header for realisation group 2, etc. If present, the "nonrealisation parameters" section should be the last section of the file.

Each line within a "realisation group" section of an ENSI specification file must begin with "parameter=", "group=" or "numreal=". (Actually, if desired, the "=" symbol can be replaced by a space. If "=" is retained, it can be surrounded by spaces.) In the first two cases, the name of a nominated parameter or parameter group respectively must follow the "=" symbol or space. This parameter or parameter group must feature in the PEST control file which defines the inverse problem. Multiple parameters and/or parameter groups can be assigned to a realisation group; in fact this will normally be the case in real-world modelling practice.

It is apparent from the above that parameter assignment to a realisation group can be done either individually using the "parameter" keyword or as a group using the "group" keyword. However a parameter cannot be assigned to a realisation group twice. Nor can a parameter be assigned to more than one realisation group. A single set of realisation factors is estimated for each realisation group. The number of such factors (equal to the number of parameter realisations) is specified following the "numreal" keyword. ("Numreal" stands for "number of realisations.")

Any adjustable parameter that is not assigned to a realisation group must be cited in the "nonrealisation parameters" section of an ENSI specification file. Only parameters (and not parameter groups) can be cited in this section. The "parameter=" string prior to the name of a featured parameter is optional in this section.

The following rules apply to the ENSI specification file and the complementary parameter uncertainty file.

1. Fixed and tied parameters retain their status as fixed and tied during the ENSI inversion process. There is no need for these parameters to be cited in an ENSI specification file. However if a tied parameter is named in a particular "realisation group" section of an ENSI specification file, then its parent parameter must be named in the same section.
2. A covariance matrix that is featured in a parameter uncertainty file cannot be associated with parameters that belong to different realisation groups.
3. The "uncertainty" of a parameter which is tied or fixed cannot be specified in a covariance matrix. If such a parameter is featured in a parameter uncertainty file, then it must be ascribed a standard deviation. This standard deviation is ignored in the ENSI inversion process.
4. As presently programmed, parameters that are featured in the "nonrealisation parameters" section of an ENSI specification file cannot be ascribed to a covariance matrix in a complementary parameter uncertainty file. These parameters must be assigned individual standard deviations. (If you feel that their uncertainties really should be specified by a covariance matrix during ENSI-based regularised inversion, then manually assign this

covariance matrix to prior information equations that pertain to these parameters in the PEST control file that ENSIPREP writes.)

5.4 ENSIPREP

5.4.1 General

ENSIPREP reads a PEST control file, a parameter uncertainty file and an ENSI specification file. It performs the following tasks.

- ENSIPREP writes a PEST control file in which new parameters are defined. These parameters are factors applied to realisations of parameters that comprise each realisation group. The total number of such parameters is equal to the number of realisation groups times the number of realisations ascribed to each group. Nonrealisation parameters are also featured in this new PEST control file. However fixed and tied parameters are not featured in this file. Nevertheless, their fixed and tied status is respected behind the scenes as the ENSI inversion process progresses.
- ENSIPREP generates random realisations of parameters to which factors are applied. These are stored (together with other information) in a binary “realisation file”. This file is read by the ENSIMOD model preprocessor. Factors are applied to these realisations prior to every model run. ENSIMOD reads these factors from a PEST-produced, text-based “realisation factor file”.
- ENSIMOD writes a template file for the ENSIMOD realisation factor file. This template file features all realisation factor parameters, as well as nonrealisation parameters (which are passed directly by ENSIMOD to the model). Hence this template file features all parameters that are cited in the new PEST control file that ENSIPREP writes.
- ENSIPREP writes a dummy realisation factor file for ENSIMOD. In this file, values of 0.0 are assigned to all realisation factor parameters. Values assigned to nonrealisation parameters are those which appear in the “parameter data” section of the PEST control file that defines the inverse problem.
- ENSIPREP writes a parameter uncertainty file that features all parameters that appear in the PEST control file that it writes; that is, it features all “ENSI parameters”. This can be used when undertaking linear parameter and predictive uncertainty analysis after the ENSI process is complete. ENSIPREP also writes a binary “uncertainty data file” that is used by the POSTENSIUNC utility. See below.

The PEST control file that is written by ENSIPREP instructs PEST to run in “regularisation” mode. Preferred-value prior information equations are ascribed to all parameters that are featured in this file. The preferred value of all factor parameters is 0.0. The preferred values of all nonrealisation parameters are their initial values as recorded in the original PEST control file. The weight that is assigned to each prior information equation pertaining to a factor parameter is calculated in the manner described above. Weights that are assigned to prior information equations pertaining to nonrealisation parameters are equal to the inverse of their prior uncertainty standard deviations as specified in the prior parameter uncertainty file.

5.4.2 Running ENSIPREP

ENSIPREP is run by typing its name at the command line. On commencement of execution, it asks a series of questions. If you answer any of these questions incorrectly, hit <Ctl-C> and then restart ENSIPREP execution.

Before asking the user any questions, however, ENSIPREP writes a message to the screen. The message is as follows.

```
Run ENSIPREP without the "/n" switch to use "regul" protocol to identify
regularisation groups.

Run ENSIPREP with the "/n" switch to use "regulgroup" protocol to identify
regularisation groups.
```

As this message implies, to commence execution of ENSIPREP, you can type either of the following commands at the command-line prompt:

```
> ensiprep
```

or

```
> ensiprep /n
```

If you type the second of these commands, then ENSIPREP uses a new protocol for identifying regularisation groups in the PEST control file that it writes. These are identified by recording the string “regulgroup” on the same line as each pertinent observation group in the “observation groups” section of the PEST control file. Otherwise the old protocol is employed. This prefixes the name of any observation group that is also a regularisation group with the string “regul”. The former protocol is usually better. ENSIPREP names regularisation groups after parameter groups, where individual parameters are being estimated; the parameter group name does not therefore need to be truncated to form the regularisation group name. Versions of PEST and PEST_HP from 18.2 onwards accept this protocol.

ENSIPREP’s first prompt is:

```
Enter name of existing PEST control file:
```

As has already been explained, this file must specify that PEST run in “estimation” mode. Also, it must contain no prior information equations. If either of these conditions are violated, ENSIPREP will cease execution with an appropriate error message.

Next ENSIPREP asks for the name of an ENSI specification file:

```
Enter name of ENSI spec file:
```

And then for the name of a parameter uncertainty file:

```
Enter name of parameter uncertainty file:
```

Next it asks for the filename base of the files that it is about to write. The prompt is:

```
Enter filename base for ENSI files:
```

Then it asks for the target measurement objective function that will appear in the “regularisation” section of the PEST control file that it is about to write:

```
Enter target measurement obj. fn. (<Enter> if very low):
```

A response of <Enter> to this prompt will result in a target measurement objective function of 1E-10. ENSIPREP’s final prompt is:

```
Enter random number seed (<Enter> if 1234):
```

A different random number seed will result in generation of different parameter realisations.

Once it has received answers to all of the above questions, ENSIPREP reads all of the files for which it has received names. It checks that information that is provided in these files is consistent and correct, and that rules that are specified above are obeyed. After it has done this, ENSIPREP generates realisations of sets of parameters; it generates as many of these realisations for each realisation group as are specified through respective “numreal” keywords in the ENSI specification file. Where the uncertainties of some parameters are described by a covariance matrix, this matrix must undergo singular value decomposition prior to random parameter set generation. ENSIPREP reports progress of the singular value decomposition and random number generation processes to the screen as it executes.

Once it has performed these tasks, ENSIPREP stores random parameter realisations in a binary realisation file named basename.rls where basename is the user-supplied filename base for ENSIPREP-generated files. This file is binary so that it can be read quickly by ENSIMOD on each occasion that the model is run. ENSIPREP then writes a new PEST control file named basename.pst in which (as described above) parameters are realisation factors; nonrealisation parameters from the original PEST control file are also featured in this new PEST control file. Each realisation factor parameter is named “*rgNfacM*” where *N* is the number of a realisation group and *M* is the factor number. *N* varies from 1 to the number of realisation groups that are featured in the ENSI specification file. *M* varies from 1 to the number of realisations ascribed to realisation group *N*.

As mentioned above, the ENSIMOD model preprocessor has two input files. One of them (the realisation file) records suites of random parameter realisations; this file is named basename.rls. The other of its input files (the realisation factor file) holds the set of factors by which these realisations are multiplied, together with the values of nonrealisation parameters. This is a text file. PEST must write this file on each occasion that it runs the model. So a template of this file is required. ENSIPREP prepares this template file, together with an exemplar realisation factor file in which the values assigned to ENSI parameters are their initial values. This input file is named basename_factors.dat while the complementary template file is named basename_factors.tpl.

In the basename.pst PEST control file that is written by ENSIPREP, the model command line is specified as *ensibatch.bat*. This is a batch/script file. ENSIPREP writes this file by modifying the original model batch file that is cited in the original PEST control file. It adds the command to run the ENSIMOD model preprocessor to the beginning of this file. Commands to delete ENSIMOD-generated files prior to running ENSIMOD are also introduced to the batch file. As is explained below, files that are written by ENSIMOD are model input files that are cited in the problem-defining PEST control file. Deletion of these files before ENSIMOD writes them is a quality assurance

measure. If, for some reason, ENSIMOD does not run, then the model which is run after ENSIMOD does not read old files, mistaking them for new ones.

Before ENSIPREP finishes execution it writes a parameter uncertainty file named *basename_prior_ensi_param.unc*. This associates a standard deviation with each newly-defined ENSI parameter. The standard deviation that is ascribed to each ENSI parameter is the inverse of the weight that is assigned to the pertinent equation in the “prior information” section of the PEST control file that it writes.

Before terminating execution, ENSIPREP writes a binary “uncertainty data file” named *basename.ucd*. This file contains data that the POSTENSIUNC program uses for calculation and listing of prior and posterior parameter uncertainties.

Once ENSIPREP has finished its work, PEST can be run using the ENSIPREP-generated PEST control file in order to undertake ENSI inversion.

5.5 ENSIMOD

5.5.1 General

ENSIMOD reads a binary file containing realisations of parameter values (i.e. a realisation file). This file is named *basename.rls*; it is written by ENSIPREP. ENSIMOD also reads a text file containing factors by which these realisations must be multiplied before being combined (i.e. added together) to form a final set of parameter values. Realisation factor multiplication is performed separately for each realisation group. This file is named *basename_factors.dat*; it is written repeatedly by PEST. For both of these files, “*basename*” is a filename base provided by the user to ENSIPREP.

Once ENSIMOD has read both of these files and performed pertinent calculations, it writes a set of model input files. It does this in the same way that PEST does. That is, it uses templates of these files. The names of template files and corresponding model input files are passed to ENSIMOD through the binary *basename.rls* input file that was written for it by ENSIPREP. These were read from the problem-defining PEST control file.

5.5.2 Running ENSIMOD

ENSIMOD is run using the following command:

```
ENSIMOD basename
```

where “*basename*” is the filename base of the files that it must read. This command is recorded in the *ensibatch.bat* file that is written by ENSIPREP. After completion of a PEST run, ENSIMOD should also be run by the user in order to obtain optimised native model parameter values. This is further discussed below.

5.5.3 Parameter Value File

As well as writing model input files, ENSIMOD writes a parameter value file on each occasion that it runs. (See Part 2 of the PEST manual for a description of this file type.) In accordance with protocols that govern SVD-assisted inversion, this file has an extension of “.bpa”; that is, it pertains to “base parameters”. These are parameters that are featured in the original, problem-defining, PEST control file that was read by ENSIPREP. The name that ENSIMOD gives to this file is *case.temp.bpa* where *case* is the filename base of the problem-defining PEST control file.

The writing of this base parameter value file by ENSIMOD allows a user to keep track of how base parameters are changing as realisation factor parameters are adjusted by PEST. However it is important to note that this file does not contain current best values of these parameters. Rather, it contains the values of base parameters that are used for the current model run (as undertaken by PEST, or by a model run agent if using PEST_HP). Depending on the purpose of the model run, these may, or may not, be close to current best parameter values. They are most likely to be close to current best base parameter values if the ENSI inversion process is progressing well, and if PEST is currently engaged in finite-difference derivatives calculation.

The manner in which optimised base parameter values can be obtained following completion of an ENSI-based PEST run is described below. To achieve this, ENSIMOD must be run directly by the modeller using the “/p” command-line switch; see below.

5.6 Some Important Workflow Elements

5.6.1 Posing the Inverse Problem

As is stated above, ENSIPREP formulates a revised inverse problem in which factors of realisations, together with the values of a few nonrealisation parameters, are estimated. Within each realisation group, each parameter realisation is multiplied by a factor; factor-weighted realisations are then summed. The sum of these factor-weighted realisations is then added to prior mean parameter values that are listed in the original PEST control file. Base parameter values that result from these actions are then recorded in model input files.

Experience to date suggests that ENSI-based solution of an inverse problem can proceed very efficiently. However problems may be encountered if prior mean parameter values are inappropriate. In certain circumstances (perhaps many circumstances) it may therefore be wise to include mean parameter values as nonrealisation parameters.

Suppose, for example, that you are estimating spatially variable hydraulic conductivities within a number of layers of a groundwater model. Suppose further that you are using families of layer-specific pilot points as your spatial parameterisation device. In a case such as this, it is good practice to define pilot point parameters as multipliers of layer-specific average hydraulic conductivity values; the handful of layer-specific mean hydraulic conductivities can then be simultaneously estimated as nonrealisation parameters. Alternatively, they could be placed into their own realisation group so that their estimation is realisation-based. In either case, separate estimation of mean and multiplier parameters keeps the latter parameters “centred”. The regularised inversion process maintains minimal departure from these central values.

The same process of separating mean parameters from parameters which describe spatial variability about these means can also be applied to boundary conditions.

Of course, there are other ways to separate “background” or “regional” hydraulic property values from local hydraulic property values. Background hydraulic property values can also be estimated individually using a limited number of “conceptual pilot points”, zones, or parameters that govern spatial trend equations. Factors can then be applied to multiplier realisations that represent spatial hydraulic property detail.

As usual, parameter estimation is an art. The way in which an inverse problem is formulated has much to do with whether (or not) success in solving that problem in a way that supports decision-making can be achieved.

5.6.2 Number of Realisations

A benefit of ENSI inversion is that the number of realisations used in solution of an inverse problem can vary between realisation groups. The modeller is responsible for the number of realisations ascribed to each group.

For any type of realisation-based inversion (including that performed by an ensemble smoother), a good rule of thumb is “the more realisations the better”. However, the numerical burden of inverse problem numerical solution rises linearly with the number of realisations. Nevertheless, certain rules apply. In general, parameter groups to which greater numbers of observations are sensitive should be endowed with a greater number of realisations. For the inverse problem as a whole, the number of realisations should be greater than the dimensionality of the inverse problem solution space. However, similar considerations apply to parameter subspaces that are defined by different realisation groups.

Suppose, for example, that layer 1 of a groundwater model contains a large number of observation wells. Then the number of realisations that is ascribed to the realisation group which contains layer 1 parameters should be greater than that ascribed to another layer in which observation wells are scarce. This strategy gives PEST the opportunity to introduce heterogeneity of sufficient detail, and in the right places, to allow model outputs to match field measurements in the first model layer. At the same time, realisations are not wasted where they are not needed (in deeper layers). Furthermore, because parameters in deeper layers are not realisation-linked to those in layer 1 as they would be in an IES-based inversion process, the ability to match layer 1 observations is not compromised by the necessity for the same realisations to match deeper layer observations.

Nevertheless, always beware of ascribing too few realisations to any particular parameter subset. The use of too few realisations prevents redundant parameter variability from cancelling over many realisations. Hence, in places/layers where data are scarce, it may be strategic to denote pertinent broadscale parameters as nonrealisation parameters.

5.6.3 Running PEST

Once ENSIPREP has built a new PEST input dataset, PEST can be run immediately. (Use PESTCHEK to verify integrity of the new PEST input dataset if you like.)

When undertaking ENSI inversion, it is often better to use PEST_HP rather than PEST. The PEST input file that is written by ENSIPREP instructs PEST to perform Broyden Jacobian updating, and sets the PEST_HP-specific UPTSTLIM control variable to 20. This increases the number of model runs required for lambda-testing. However this can pay dividends in terms of efficient objective function reduction in highly nonlinear cases. So the investment in extra model runs often provides a high return.

When writing its ENSI-based PEST control file, ENSIPREP assigns default values to many PEST control variables. Feel free to alter the values of these variables if you wish. To eschew regularisation, remove it from the ENSIPREP-generated PEST control file using the PEST SUBREG1 utility. If your model is a little unstable, you may wish to alter the values of PEST control variables that govern

finite-difference derivatives calculation; these occupy the “parameter groups” section of the PEST control file. Observation weights can, of course, be changed at will, as can weights that ENSIPREP ascribes to prior information equations. Note that ENSIPREP sets the value of the IREGADJ regularisation control variable to zero. Automatic assignment of relative inter-regularisation group weighting by PEST is therefore disabled. Set this to 1 (or another value of your choice) if you think that this is more appropriate for your particular inversion problem.

In accordance with normal PEST behaviour, at any time during the ENSI inversion process best parameter values are listed in a parameter value file named *basename.par* where *basename* is the filename base of the PEST control file that is written by ENSIPREP and run by PEST. ENSIPREP also invokes the PARSAVEITN control variable in the *basename.pst* PEST control file that it writes. Hence parameter value files are recorded during each iteration of the ENSI inversion process; these files are named *basename.par.N* where *N* is the iteration number. Unfortunately, these parameter values files contain values of realisation factor parameters and not base parameters. Only the latter are useable by a modeller.

As is discussed above, file *case.temp.bpa* contains values of base parameters that pertain to the latest model run undertaken by PEST itself, or by one of its agents. This allows a user to monitor how parameters are changing as the inversion process progresses.

5.6.4 After Running PEST

Once the inversion process is complete, the easiest way to obtain best base parameter values is to run ENSIMOD yourself by typing the following command:

```
ensimod basename /p
```

When run using this command, ENSIMOD reads file *basename.par*. Presumably this contains optimised ENSI parameter values. Many of these pertain to realisation factors. (Copy an iteration-specific *basename.par.N* parameter value file to *basename.par* if you would rather that ENSIMOD read that parameter value file instead of *basename.par*.) ENSIMOD then writes a parameter value file named *case.bpa* instead of *case.temp.bpa*, where *case.pst* is the name of the original, problem-defining PEST control file that was read by ENSIPREP. This parameter value file contains optimised base parameter values.

Alternatively, run ENSIMOD using the following command:

```
ensimod basename /p parfile
```

where *parfile* is the name of a parameter value file that contains ENSI parameter values, and is therefore compatible with files such as *basename.par*.

Once this parameter value file has been written, the PARREP utility can be used to populate a new base PEST control file with optimised parameter values. At the same time, the NOPTMAX control variable in this new file can be set to zero so that PEST runs the model only once and then records the objective function that it calculates before ceasing execution. You can confirm that this objective function is what you expect it to be from the previous inversion process. The command to run PARREP in this context is:

```
parrep case.bpa case.pst case_soln.pst 0
```

where case is the filename base of the original PEST control file that was read by ENSIPREP. (You can name the PEST control file that PARREP writes anything that you like; it does not need to be case_soln.pst as in the above example.) PEST can now be run using the command:

```
pest case_soln
```

As stated above, PEST will run the model once and then cease execution. (Make sure to include “0” in the above PARREP command line to ensure that NOPTMAX is set to zero in the new PEST control file.) After this has happened, all model input files will be populated using optimised parameters. The objective function that PEST records on the screen should be the same as the measurement objective function attained during the previous ENSI inversion process. Hopefully, model outputs exhibit a good fit with field measurements.

As a further alternative, ENSIMOD can be run using the following command:

```
ensimod basename /c csvfile1 csvfile2
```

When run in this way, ENSIMOD reads a CSV file (named *csvfile1* in the above prompt). It writes another CSV file (named *csvfile2* in the above prompt). The following protocols are assumed for this file:

- The first line contains column headers. For all but the first column, these headers are the names of ENSI parameters. These should be arranged in the same order as those in the PEST control file that was written by ENSIPREP.
- The following lines contain realisations of ENSI parameters.

Such a file will be written by PESTPP-IES if it is used in conjunction with an ENSIPREP-produced PEST control file. You may wish to do this instead of using PEST_HP to history-match a model. More likely, you may decide that, after PEST_HP has calibrated the model, you would like to undertake the following workflow:

- Use PREDUNC7 to calculate a posterior covariance matrix for ENSI parameters (see below);
- Use this matrix as a prior uncertainty file for PESTPP-IES, together with a PEST control file that contains calibrated ENSI parameters (see below);
- Use PESTPP-IES to obtain posterior realisations of ENSI parameters.

This workflow provides a means of rapidly sampling the posterior probability distribution of ENSI parameters. What are of interest, however, are native model parameters. A CSV file of realisations of native model parameters can be obtained from a CSV file of ENSI model parameters using ENSIMOD in the above way.

5.7 Posterior Linear Uncertainty Analysis

5.7.1 General Linear Analysis

The PEST control file that is written by ENSIPREP is like any other PEST control file. (We refer to this PEST control file as the “ENSI PEST control file” to distinguish it from the base PEST control file.) As ENSI inversion progresses, sensitivities of model outputs to ENSI parameters are recorded in a complementary ENSI Jacobian matrix file (i.e. JCO file). Recall that parameters that populate the basename.pst PEST control file (i.e. the ENSI PEST control file that was written by ENSIPREP), and for

which sensitivities are recorded in the complementary *basename.jco* Jacobian matrix file, are a little “esoteric”. Most of them, if not all of them, are realisation factor parameters. The specifics of these realisations (that is, the base parameter values that they encapsulate) are determined by the random number seed employed by ENSIPREP when it generated these realisations; hence they are somewhat arbitrary.

The esoteric nature of ENSI parameter sensitivities does not preclude their use in linear uncertainty analysis. If the sensitivities of a prediction to these same ENSI parameters are available, then the posterior uncertainty of that prediction can be obtained using the PREDUNC1 utility. These sensitivities are available automatically if a prediction is being “carried” as a zero-weighted observation in the base PEST control file; recall that all observations that are featured in the base PEST control file are transferred to the ENSI PEST control file. If this is not the case, then a new ENSI PEST control file can be built which uses the same parameters but features “observations” that are actually predictions. Set the NOPTMAX control variable to -1 or -2 in this file to calculate a predictive ENSI Jacobian matrix. Alternatively, build a new base PEST control file in which “prediction observations” replace history-matching observations. Then run ENSIPREP again to build a predictive ENSI PEST control file which can then be used to calculate a predictive Jacobian matrix. When running ENSIPREP in this fashion, use the same random number seed that you used in the original ENSIPREP run.

Prior parameter uncertainties (required for calculating posterior parameter uncertainties) are available from the *basename_prior_ensi_param.unc* uncertainty file written by ENSIPREP.

5.7.2 Posterior Parameter Uncertainties: A Simple Workflow

A simple workflow is available for evaluating approximate posterior uncertainties of native model parameters, and for comparing these posterior uncertainties with their prior uncertainties. The ability (or otherwise) of the history-matching process to reduce these uncertainties can then be gauged.

First, calibrate the model using ENSI inversion in the manner outlined above. As has already been explained, at the end of the ENSI inversion process, sensitivities of model outputs to ENSI parameters are recorded in an ENSI JCO file named *basename.jco*. This Jacobian matrix can form the basis for linear uncertainty analysis. Alternatively, use the PARREP utility to populate a new ENSI PEST control file with optimised ENSI parameters. Then set NOPTMAX to -1 or -2 in this file and calculate an ENSI Jacobian matrix using optimised ENSI parameters.

PREDUNC7 can now be used to calculate a posterior covariance matrix of ENSI parameters. However before you do this, you must remove regularisation from the ENSI PEST control file. Use SUBREG1 to do this. Then use the JCO2JCO utility to obtain a complementary JCO file from the ENSI JCO file that includes regularisation.

If you like, you can use the PWTADJ2 utility to make weights “right” in this new ENSI PEST control file. But this is not essential.

Now you can use the PREDUNC7 utility to obtain a posterior ENSI parameter covariance matrix and uncertainty file. (All you need is the covariance matrix file; the PREDUNC7-generated uncertainty file simply cites the covariance matrix file.) When PREDUNC7 asks for the reference variance, calculate

this as the (measurement) objective function divided by the number of non-zero-weighted observations. When PREDUNC7 asks you for a prior covariance file for ENSI parameters, provide the name of the file that ENSIPREP wrote for this purpose. It is named basename_prior_ensi_param.unc.

So now we have a posterior covariance matrix for ENSI parameters. On its own, this is of limited use to us, for it is the posterior uncertainties of native (i.e. base) model parameters that we seek (or an approximation thereto), and not the posterior uncertainties of ENSI parameters. What we now have, in fact, is the posterior covariance matrix of the **f** parameters of equation 5.3. We will refer to this matrix as $C'(\mathbf{f})$ (where the prime signifies posterior). What we need, however, is the posterior covariance matrix of the **j** vector that is featured in this equation, i.e. $C'(\mathbf{j})$. From equation 5.3 this can be calculated as:

$$C'(\mathbf{j}) = \mathbf{K}C'(\mathbf{f})\mathbf{K}^T \quad (5.7)$$

It is important to note that the covariance matrix of **j** calculated in this way is approximate and rank-deficient. So it should not be used as a basis for random field generation. However it can still provide a useful indication of history-match-engendered parameter uncertainty reduction.

The POSTENSIUNC (for “posterior ENSI uncertainty analysis”) utility calculates the diagonal elements of $C'(\mathbf{j})$. That is, it calculates approximations to the posterior variances of native model parameters; posterior standard deviations are also calculated by taking the square roots of these. POSTENSIUNC compares these with prior native model parameter variances and standard deviations. The latter were recorded by ENSIPREP in a binary “uncertainty data file” which was written specifically for the use of POSTENSIUNC.

5.7.3 Running POSTENSIUNC

POSTENSIUNC should be run from the same folder as that from which ENSIPREP was run so that it has access to the files that were written for it by ENSIPREP. It should be run after the ENSI inversion process is complete, and after the PREDUNC7 utility has been used to obtain a posterior covariance matrix of ENSI parameters (i.e. $C'(\mathbf{f})$).

Upon commencement of execution, POSTENSIUNC asks:

```
Enter filename base of ENSIPREP-produced files:
```

POSTENSIUNC reads two files that were written for it by ENSIPREP when the latter program was employed to set up the ENSI inversion process. The first of these files is named basename.ucd while the second is named basename.rls; both of them are binary files. The first of these files is an “uncertainty data file”. The second is the realisation file that is also used by ENSIMOD.

Next POSTENSIUNC asks:

```
Enter name of PREDUNC7 posterior ENSI covariance matrix file:
```

This file is written by PREDUNC7 using the steps described above. (Note that you should provide the name of the covariance matrix file that was written by PREDUNC7 and not the name of the uncertainty file that cites the covariance matrix file.)

POSTENSIUNC next reads the PREDUNC7-generated posterior ENSI covariance matrix file, ensuring that its contents are compatible with those of the other files that it reads. It then asks for the name of the file that it must write:

```
Enter name for posterior native param uncert table file:
```

After performing the calculations that are encapsulated in equation 5.7, POSTENSIUNC records a tabular file comprised of 6 columns. These columns are as follows:

- names of adjustable (i.e. not fixed or tied) native model parameters;
- their transformation status (“none” or “log”);
- prior parameter variances;
- prior parameter standard deviations;
- posterior parameter variances;
- posterior parameter standard deviations.

As stated above, because of the rank-deficient nature of the matrix that is calculated using equation 5.7, posterior native parameter uncertainties are only approximate. In fact, it is possible that for some parameters, calculated posterior uncertainties will be slightly greater than prior uncertainties. This may be the case for parameters that are constrained to only a small extent by the calibration dataset. This is an outcome of the rank-deficient nature of $C'(j)$.

Nevertheless, in spite of their shortcomings, the numbers that are listed in the POSTENSIUNC-generated table can have high didactic worth. Plots of parameter uncertainty reduction can be graphed in illustrative bar charts and/or mapped over a model domain. Despite their approximate nature, they can provide a graphic illustration of history-match-enabled flow of information from observations to parameters.

5.8 Summary of Workflow

5.8.1 General

We now briefly summarise the entire ENSI workflow for model calibration and appraisal of parameter uncertainty reduction achieved through history-matching.

1. Set up a PEST input dataset in the normal way.
2. Prepare a parameter uncertainty file and an ENSI specification file.
3. Use ENSIPREP to prepare a PEST input dataset for ENSI inversion.
4. Use PEST or PEST_HP to calibrate the model using this file. If necessary, stop the inversion process yourself when you think that the measurement objective function has fallen far enough.
5. Run ENSIMOD using the “/p” switch to build a BPA file containing optimised base parameters.
6. Use the PARREP utility to create a new base PEST control file in which initial parameter values are optimised parameter values.

To obtain a table of prior and posterior uncertainties for base model parameters, add the following steps to the workflow.

7. Use the SUBREG1 utility to remove regularisation from the ENSIMOD-generated PEST control file on which the previous inversion exercise was based, building a new ENSI PEST control file in the process.
8. Use JCO2JCO to obtain a complementary Jacobian matrix file based on the JCO file that was produced during the inversion process. (This new JCO file has regularisation prior information sensitivities removed.)
9. Employ PREDUNC7 to compute a posterior ENSI parameter covariance matrix. When asked for the prior uncertainty file for ENSI parameters, cite the uncertainty file that was prepared by ENSIPREP. When asked for the reference variance, divide the measurement objective function by the number of non-zero-weighted observations.
10. Use POSTENSIUNC to build a table of prior and posterior uncertainties for native (i.e. base) model parameters.
11. Plot or graph these as appropriate.

See also the description of program ENSIMOD above. If you conduct PESTPP-IES-based nonlinear uncertainty analysis using an ENSI model after calibrating that model using PEST_HP, ENSIMOD provides the means to convert a CSV file containing IES-adjusted ENSI model realisations to a CSV file containing realisations of native model parameters.

5.8.2 Final Words

The number of model runs that ENSI requires to fill its Jacobian matrix (N_r) can be calculated as follows:

$$N_r = \sum_i^R N_i + N_p \quad (5.8)$$

Where R is the number of realisation groups, N_i is the number of realisations allocated to realisation group i and N_p is the number of nonrealisation parameters. If the UPTTESTMIN control variable is set to 20 and Broyden Jacobian updating is turned on, then another 40 model runs are required to complete an iteration. When the objective function does not fall much on any iteration, then PEST_HP may switch to central derivatives calculation. In this case N_r is doubled. Keep this in mind when reflecting on your model run budget.

There is a temptation to define lots of realisation groups. It is early days yet, but my advice is to be wary about this, perhaps limiting the number of realisation groups to five at the most. A realisation group to which many observations are sensitive should be allocated many realisations. In fact, it should be allocated at least as many realisations as there are separate pieces of information in the observation dataset that inform the parameter types that are assigned to that realisation group. If this is not done, then model-to-measurement fits will not be good, or other parameter types will attempt to harvest the information that is meant for the correct parameter type, this introducing distortions to the calibrated parameter field. Having said this, where parameters of very different types are likely to be the recipients of information from very different sources, then it is probably better to place them into different realisation groups. However each group must have enough realisations to receive this information.

So the number of realisation groups, and the number of realisations assigned to each group, is a trade-off. Your guide in selecting the number of realisation groups, and the parameters to assign to each realisation group, is intuition. However the likelihood that a particular parameter group will be

populated with an insufficient number of realisations to receive the information that it must harvest rises with the number of realisation groups that you define.

There is one final practical matter to consider. Make sure that the width of parameter spaces on PEST template files is 14 characters or more (if the model allows it). As is explained in the PEST manual, when PEST writes a number to a model input file, it can adjust its internal representation of the number to accord with the external representation of that same number if the parameter space isn't very wide. (Significant figures are important when parameters are subjected to small differences when undertaking finite-difference derivatives calculation.) However PEST_HP cannot make this adjustment when model input files are written by ENSIMOD. The integrity of finite difference derivatives calculation therefore depends on an ability to write parameters to model input files with maximum precision, and to read observations from model output files with maximum precision.

5.9 PCA Inversion

5.9.1 General

It is easy to undertake PCA inversion using virtually the same algorithm, and an almost identical workflow, as that which is required for ENSI inversion. As is discussed extensively above, when undertaking ENSI inversion, factors are applied to realisations of parameters (i.e. ensembles of parameters) drawn from subsets of the prior parameter probability distribution. Different factors can be estimated for different groups of parameters which have different prior covariance matrices. The values of some individual parameters can be estimated simultaneously.

When undertaking PCA inversion, factors are applied to eigencomponents of prior parameter covariance matrices rather than to random realisations of parameters that are drawn from these matrices. A parameter covariance matrix has as many eigencomponents as there are parameters that are featured in the matrix. However, you do not need to estimate factors that are applied to all of these eigencomponents. In fact, the fewer eigencomponent factors that are estimated, the greater is the model run efficiency of the inversion process. This is because the inversion process works in a subspace that is spanned by only those eigencomponents for which factors are estimated. Often, this does not compromise the ability of the inversion process to attain a good fit with field observations. This is because high order eigencomponents (corresponding to high singular values of a parameter covariance matrix) express broadscale hydraulic property variability while low order eigencomponents (which correspond to low singular values of a parameter covariance matrix) express hydraulic property detail that is often beyond the ability of the inversion process to infer, and hence lies within the calibration null space.

An advantage of using eigencomponents of prior covariance matrices rather than realisations drawn from prior parameter probability distributions as a basis for dimensional reduction, is that the calibrated parameter field achieved using the former option is likely to be smoother than that which is achieved using the latter option. This harmonises with the role of the calibration process, which is to attain the simplest parameter field that fits a calibration dataset; the simpler is this parameter field, the more likely it is to be that of minimized error variance.

5.9.2 Theory

We repeat equation 5.2 in which a model parameter set $\mathbf{j}$ is formulated through use of estimable factors $\mathbf{f}_i$ applied to ensembles of parameter subsets.

$$\mathbf{j} = \begin{bmatrix} \mathbf{K}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{K}_3 \end{bmatrix} \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_3 \end{bmatrix} \quad (5.2)$$

Recall that columns of the $\mathbf{K}_i$ matrices that appear in equation 5.2 are random realizations of subsets of parameters. Suppose that the covariance matrix pertaining to the i^{th} subset of parameters is $\mathbf{C}_i(\mathbf{k})$. Let us now subject $\mathbf{C}_i(\mathbf{k})$ to singular value decomposition, so that:

$$\mathbf{C}_i(\mathbf{k}) = \mathbf{V}_i \mathbf{S}_i \mathbf{V}_i^T \quad (5.9)$$

When undertaking PCA inversion, equation (5.2) becomes:

$$\mathbf{j} = \begin{bmatrix} \mathbf{V}_1 \mathbf{S}_1^{1/2} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{V}_2 \mathbf{S}_2^{1/2} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{V}_3 \mathbf{S}_3^{1/2} \end{bmatrix} \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_3 \end{bmatrix} \quad (5.10)$$

Note the following:

- In equation 5.10, singular value truncation takes place at the number of “realisations” that is assigned to any realisation group through the ENSI specification file (see below). So only as many columns of $\mathbf{V}_i$ are represented as the number of user-specified realisations.
- ENSIPREP allows you to combine ensemble-based inversion with PCA-based inversion in a single inversion process. Hence equations 5.2 and 5.10 can be combined; for example:

$$\mathbf{j} = \begin{bmatrix} \mathbf{K}_1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{V}_2 \mathbf{S}_2^{1/2} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{V}_3 \mathbf{S}_3^{1/2} \end{bmatrix} \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_3 \end{bmatrix} \quad (5.11)$$

The prior uncertainties of $\mathbf{f}_i$ vectors that are used in ensemble-based ENSI inversion are discussed in section 5.2.3. Recall equation 5.6:

$$\mathbf{C}_i(\mathbf{f}) = \frac{\mathbf{I}}{N-1} \quad (5.6)$$

where N is the number of realisations ascribed to the i^{th} realisation group.

Where factors are applied to the columns of $\mathbf{V}_i \mathbf{S}_i^{1/2}$ (i.e. to covariance matrix eigenvectors instead of parameter realisations), equation 5.6 becomes:

$$\mathbf{C}_i(\mathbf{f}) = \mathbf{I} \quad (5.12)$$

5.9.3 Setup

The workflow for PCA inversion, or mixed inversion where some factors are applied to some random parameter realizations and some factors are applied to parameter covariance eigenvectors, is almost identical to that described in previous parts of this chapter for ensemble-based ENSI inversion. The only change that is required to the workflow is a slight alteration to the ENSI specification file. Figure 5.2 is altered from Figure 5.1; alterations are highlighted.

```

* realisation group 1
pargroup=k1
pargroup=k2
pargroup=k3
numreal=120
parameter=ghbcond1
parameter=ghbcond2

* realisation group 2
pargroup=por1 pca
pargroup=por2 pca
pargroup=por3 pca
numreal=100

* nonrealisation parameters
p_global
k_global

```

Figure 5.2 An ENSI specification file in which PCA factors are estimated for some parameters.

As is apparent from Figure 5.2, the only alteration to the ENSI input dataset that is described above is that the string “pca” must follow the names of parameter groups for which PCA inversion is required.

The following rules apply. (ENSIPREP will inform you if these rules are violated.)

1. The string “pca” cannot follow the name of a parameter. It must follow the name of a parameter group.
2. One or a number of covariance matrices must be associated with any parameter group which precedes the “pca” string.
3. If a parameter group within a realisation group is designated as “pca”, then all parameter groups that are assigned to the same realisation group must be designated as “pca”. That is the whole realisation group is effectively a “pca realisation group”.
4. If a parameter group within a realisation group is designated as “pca”, then no individual parameters must be assigned to that realisation group.
5. The number of realisations ascribed to a realisation group must not exceed the number of parameters within any parameter group that is assigned to the realisation group. Nor must it exceed the number of parameters assigned to any covariance matrix that is assigned to any subset of a parameter group (if more than one covariance matrix is assigned to a single parameter group). The reason for this is simple. There are only as many eigencomponents in a covariance matrix as the number of parameters that are featured in that matrix. If there are more realisations than eigencomponents, ENSIPREP runs out of eigencomponents. (If you inadvertently ask for too many realisations, ENSIPREP will inform you of your error.)

When implementing PCA inversion, it helps with personal book-keeping if there is a one-to-one relationship between parameter groups and covariance matrices. That is, try to avoid assigning different covariance matrices to different subsets of parameters within a single parameter group. Assign parameters with different covariance matrices to different parameter groups instead. And, if possible, assign different parameter groups to different realisation groups. (Let me know if ENSIPREP needs to accommodate more realisation groups.) Parameters are usually divided into different

groups because they differ in some way from each other. PCA-based inversion will nearly always work better if these differences are respected. Hence, if possible, assign different covariance matrices to different parameter groups, and different parameter groups to different realisation groups. Note, however, that ENSIPREP does not insist on this. (This matter is further discussed below.)

5.9.4 The Workflow

All other elements of the ENSI workflow that are described above are the same for PCA-based inversion as they are for ensemble-based ENSI inversion. These include the use of PEST/PEST_HP, ENSIMOD and POSTENSIUNC.

5.9.5 Which to Use?

The model run efficiency of ensemble-based ENSI inversion and PCA-based ENSI inversion depends on the number of realisations that are assigned to each realisation group. When undertaking PCA inversion, the “number of realisations” actually defines the point at which eigencomponents are truncated. As stated above, truncated eigencomponents express detail, and have small singular values. Expression of this detail may not be necessary to attain a good fit with a calibration dataset.

Ideally, therefore, the run time efficiency of both approaches to ENSI inversion should be about the same. However, PCA inversion should provide smoother parameter fields than ensemble-based inversion. But will the absence of “bumps and lumps” in the calibrated parameter field (some of which may be spurious) compromise the ability of the inversion process to provide a good fit with some aspects of field data? The jury is out. Let me know of your experiences.

5.9.6 Efficiency vs Bias

We conclude this chapter with an example of PCA inversion design. It is worth noting, however, that while conclusions that are drawn from this example are directly applicable to PCA-based ENSI inversion, they also have repercussions for ensemble-based ENSI inversion, as well as for ensemble-based inversion in general, including that conducted using an ensemble smoother such as PESTPP-IES. We focus on the potential for parameter bias that may emerge from pursuit of model run efficiency.

Suppose that we are calibrating a three-layer groundwater model. In this discussion, we focus only on the hydraulic conductivity of these three model layers. Suppose further that the first and second model layers possess 200 parameters while the third model layer possesses 50 parameters (possibly because of a diminished areal coverage compared to layers that overlie it.) Suppose further that each of these sets of layer-specific parameters is assigned to its own parameter group. We name these groups *klay1*, *klay2* and *klay3*. We further suppose that (as is normally the case) the covariance matrix ascribed to each of these parameter groups is independent of that assigned to the other groups. We thereby assume that model layer hydraulic conductivities are statistically independent of each other. If we were to build a joint hydraulic conductivity covariance matrix from individual, layer-specific hydraulic conductivity covariance matrices, it would therefore be block diagonal. This is schematized in Figure 5.3.

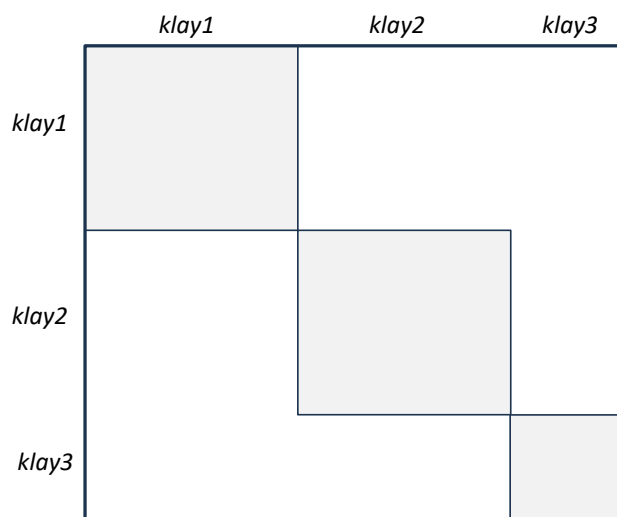


Figure 5.3. Schematic of the composite hydraulic conductivity covariance matrix. Element values are zero in unshaded parts of this matrix.

A modeller may decide to assign all hydraulic conductivity parameters to a single realisation group, and to employ PCA as a basis for their estimation. The pertinent fragment of the ENSI specification file may therefore appear as in Figure 5.4.

```
* realisation group 2
pargroup=klay1  pca
pargroup=klay2  pca
pargroup=klay3  pca
numreal=100
```

Figure 5.4. Part of an ENSI specification file.

Unfortunately, this approach to inversion is problematical. The first problem is that ENSIPREP will object to 100 realisations being assigned to this realisation group. This is because it exceeds the number of parameters in the *klay3* parameter group. There are only 50 eigencomponents in the covariance matrix that is assigned to this parameter group. As is discussed above, the number of realisations that is assigned to a PCA realisation group cannot exceed the number of parameters that is associated with the smallest covariance matrix that is ascribed to any parameter group that is featured in the realisation group.

If the number of realisations ascribed to realisation group 2 is reduced to 50, then ENSIPREP will no longer complain.

However another, more subtle, problem then arises. As posed above, the inversion process will seek factors that are applied to covariance matrix eigencomponents; these factors are elements of the vector $\mathbf{f}$ that appears in equations that are presented earlier in this chapter. Let us designate the first of these factors as f_1 ; this is the first element of the vector $\mathbf{f}$. Factor f_1 is applied to the first eigencomponent of the *klay1* covariance matrix as well as the first eigencomponent of the *klay2* covariance matrix together with the first eigencomponent of the *klay3* covariance matrix. Likewise f_2 is applied simultaneously to the second eigencomponents of all of these matrices. These early eigencomponents encapsulate broadscale hydraulic property variability in their respective model layers. Obviously, the factors that are applied to this broadscale variability will have a large effect on the level of fit that can be achieved with members of the calibration dataset that pertain to each of

these model layers. Unfortunately, however, it may not be possible to attain a good fit with all members of the calibration dataset unless different factors are applied to eigencomponent-described spatial variability of hydraulic properties in different model layers.

There are two ways to fix both of the above problems.

The first is to provide a joint covariance matrix for all members of the *klay1*, *klay2* and *klay3* parameter groups. As is apparent from Figure 5.3, this matrix is block diagonal. (Construction of this multi-layer covariance matrix may be undertaken using, for example, the PPCOV utility from the PEST Groundwater Utility Suite, with pilot points in different layers assigned to different zones.) When ENSIPREP subjects this block-diagonal covariance matrix to singular value decomposition (SVD), each of its eigencomponents will be layer-specific. That is, each of them will describe variability in one of the blocks that appears in Figure 5.3 while having zero-valued elements outside of these blocks. However, because the SVD process orders eigencomponents according to magnitude of corresponding singular values, the eigencomponent matrix that emerges from SVD of the joint covariance matrix will not, in fact, be block-diagonal. Instead, some of the eigencomponents pertaining to all three model layers will be associated with high singular values, and will therefore occupy early columns of the joint eigencomponent matrix. Calibration-adjustable, layer-specific factors will therefore be applied to all of these layer-specific eigencomponents. Furthermore, if desired, up to 250 factors can be estimated by assigning up to 250 realisations to the pertinent realisation group in the ENSI specification file. Collectively, these 250 eigencomponents encapsulate all spatial parameter variability that can occur within all of these three layers. At the same time, each of the 250 estimated factors is applicable to a single layer because of the layer-specific nature of each joint covariance matrix eigencomponent. (In practice, fewer realisations than this would probably encompass enough hydraulic property variability to ensure a good fit with a calibration dataset.)

The second (and easier) means to avoid the above problems is to assign each of the layer-specific *klay1*, *klay2* and *klay3* parameter groups to different realisation groups in the ENSI specification file. Each of these groups can then maintain its own layer-specific covariance matrix. This strategy also allows individual factors to be applied to individual, layer-specific covariance matrix eigencomponents. See Figure 5.5.

```
* realisation group 2
pargroup=klay1  pca
numreal=50

* realisation group 3
pargroup=klay2  pca
numreal=50

* realisation group 4
pargroup=klay3  pca
numreal=25
```

Figure 5.5. A revised ENSI specification file.

This second strategy requires the use of more realisation groups than the first strategy. Perhaps it will also require the use of a greater total number of realisations (and hence model runs per iteration) than the first strategy, as each realisation group must possess enough realisations to

express hydraulic property spatial variability that is required to achieve a good match with those aspects of the calibration dataset that are sensitive to individual layer properties.

Assigning the “correct” number of realisations to each realisation group is indeed guesswork to some extent. A modeller may indeed request a greater number of total realisations (and hence model runs per iteration) where an ENSI specification file possesses a greater number of realisation groups. However use of a greater number of realisation groups enables the unlinking of different parameter types. This grants greater flexibility to the inversion process as it adjusts parameters in order to receive information that is resident in a calibration dataset. This reduces the chances of biasing parameters as they are adjusted in order to match observations that may be artificially sensitive to them purely because they are arbitrarily linked to other parameters that are actually sensitive to them.

One way or another, there is always a price to be paid for model-run efficiency.

This simple example demonstrates how working in a reduced dimensional parameter subspace may induce parameter (and therefore possibly predictive) bias. The reason for this bias is clear when undertaking PCA-based ENSI inversion. However the same principles apply when undertaking ensemble-based ENSI inversion; however they are not quite as obvious because dimensions in parameter space are defined by random parameter realisations rather than by parameter covariance matrix eigencomponents. The same logic applies to IES, where parameters of many different types are encapsulated in a relatively small number of calibration-adjustable realisations; in each of these realisations parameters of different types are linked in immutable ratios.

The use of ENSI (whether ensemble-based or PCA-based) provides a modeller with the means to decouple parameters to some extent, and to gain greater freedom in trading model run efficiency against the propensity for calibration-induced parameter/predictive bias.

6. References

Bayer, P., de Paley, M. and Bürger, C.M., 2010. Optimization of high-reliability-based hydrological design problems by robust automatic sampling of critical model realizations. *Water Resour. Res.* 46, doi:10.1029/2009WR008081.

Bayer, P., Bürger, C.M. and Finkel, M., 2008. Computationally efficient stochastic optimization using multiple realizations. *Adv. Water Resour.* 31(2), 399-417. doi:10.1016/j.advwatres.2007.09.004.

Bárdossy, A. and Hörning, S., 2016. Random mixing: a approach to inverse modelling for groundwater flow and transport problems. *Transp. Porous Med* 114:241-259.

Chada, N.K., Iglesias, M.A., Roininen, L. and Stuart, A.M., 2018. Parameterizations for ensemble Kalman Inversion. *Inverse problems*, 34, 055009 (31pp).

Doherty, J., 2015. Calibration and uncertainty analysis for complex environmental models. Published by Watermark Numerical Computing, Brisbane, Australia. 227pp. ISBN: 978-0-9943786-0-6. Downloadable from www.pesthomepage.org.

Haese, B., Hörning, S., Chwala, C., Bárdossy, A., Schalge, B. and Kuntzmann, H., 2017. Stochastic reconstruction and interpolation of precipitation fields using combined information of commercial microwave links and rain gauges. *Water Resources Research*. DOI 10.1002/2017/WR021015.

Iglesias, M.A., Law, K.J.H. and Stuart, A.M., 2013. Ensemble methods for inverse problems. *Inverse problems*, 39, 045001 (20pp).

Raanes, P. N., Stordal, A. S., and Evensen, G., 2019 Revising the stochastic iterative ensemble smoother, *Nonlin. Processes Geophys. Discuss.*, <https://doi.org/10.5194/npg-2019-10>, in review, 2019.

White, J.T., 2018. A model-independent iterative ensemble smoother for efficient history matching and uncertainty quantification in very high dimensions. *Environmental Modeling and Software*, 109. doi: 10.1016/j.envsoft.2018.06.009.